



Universidad de Valladolid

**Escuela de Ingeniería Informática de
Valladolid**

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática

Mención en Ingeniería de Software

Título del TFG

Autor:

D. Sergio Alcón González

Tutores:

**D. Luis Ignacio Jiménez Gil
D. Mario Corrales Astorgano**

Resumen

Palabras clave:

Abstract

Keywords: ...

Índice general

1	Introducción	5
1.1	Contexto y motivaciones	5
1.2	Objetivos	6
1.3	Metodología	6
1.4	Estructura del documento	7
2	Antecedentes y Estado del Arte	9
2.1	Evolución de los juegos de estrategia	9
2.2	El juego original: <i>Stratego</i>	11
2.3	Variaciones, adaptaciones y videojuegos relacionados	12
2.4	Motores de desarrollo de videojuegos	13
3	Concepto del videojuego	21
3.1	Resumen del juego	21
3.2	Mecánicas	22
3.2.1	Tablero	22
3.2.2	Colocación inicial	22
3.2.3	Piezas	23
3.2.4	Turnos	23
3.2.5	Resolución de combates	24
3.2.6	Condiciones de victoria	25
3.2.7	Interacción y control	25
3.3	Ambientación	25

3.4	Diseño de niveles	27
3.5	Audiencia	27
4	Planificación	29
4.1	Planificación inicial	29
4.1.1	Iteración 1: Implementación del juego base	29
4.1.2	Iteración 2: Interfaz de usuario y pulido visual	30
4.1.3	Iteración 3: Mejoras audiovisuales	31
4.1.4	Iteración 4: Implementación del bot inteligente	31
4.1.5	Iteración 5: Pulido final	32
4.2	Riesgos	35
4.3	Restricciones	36
4.4	Herramientas	36
4.5	Variación de planificación	37
4.6	Costes del proyecto	38
5	Análisis	41
5.1	Actores y roles	41
5.2	Modelado de Requisitos	41
5.3	Casos de uso	44
5.3.1	Lista de casos de uso	44
5.4	Modelo de dominio	49
5.5	Modelo de proceso de negocio	50
5.5.1	Actividad: Jugar partida (CU-01)	50
5.5.2	Actividad: Jugar turnos (CU-02)	50
5.5.3	Actividad: Resolver combate (CU-03)	52
5.5.4	Actividad: Consultar reglas (CU-04)	53
6	Diseño	55
6.1	Arquitectura del sistema	55

6.2	Módulos o componentes del sistema	59
6.3	Diseño de clases	61
6.4	Diseño de interfaz de usuario	61
6.5	Patrones de diseño aplicados	61
6.6	Seguridad y rendimiento	61
7	Implementación	63
7.1	Organización del código	63
7.2	Licencias requeridas	63
7.3	Batería de pruebas	63
8	Conclusiones y Líneas futuras	65
A	Manual de instalación	67
B	Manual de usuario	69
C		71
D		73
	Bibliografía	75

Índice de figuras

2.1	Tablero del Juego Real de Ur [1]	15
2.2	Tablero del juego Senet [2]	16
2.3	Tablero del juego Dou Shou Qi [3]	17
2.4	Tablero del juego L'attaque [4]	18
2.5	Tablero del juego Luzhanqi [5]	19
3.1	Esquema del tablero	22
4.1	Diagrama de Gantt general del desarrollo del TFG, incluyendo todas las iteraciones y la documentación	34
5.1	Diagrama de casos de uso	44
5.2	Diagrama del modelo de dominio	49
5.3	Diagrama de actividad: Jugar partida (CU-01)	50
5.4	Diagrama de actividad: Jugar turnos (CU-02)	51
5.5	Diagrama de actividad: Resolver combate (CU-03)	52
5.6	Diagrama de actividad: Consultar reglas	53
6.1	Estructura de la escena <code>Tile</code> en el editor de Godot	56
6.2	Modelo de ejecución de Godot ilustrado con algunos nodos del proyecto.	57
6.3	Vista simplificada de la arquitectura de NeuroForge en tres capas.	58
6.4	Ejemplo de comunicación entre capas durante la confirmación del despliegue.	59
6.5	Módulos del sistema y sus dependencias principales.	60

Índice de tablas

2.1	Comparativa entre motores de desarrollo de videojuegos	15
4.1	Iteraciones planificadas, estimación de horas y fechas de finalización	33
4.2	Riesgos identificados y planes de contingencia	35
4.3	Herramientas	37
4.4	Desglose de costes estimados del proyecto en contexto profesional	40
5.1	Lista de casos de uso del sistema	44
5.2	CU-01: Jugar partida	45
5.3	CU-01: Jugar partida	46
5.4	CU-03: Resolver combate	47
5.5	CU-04: Consultar reglas	48

Agradecimientos

Glosario

ACRONIMO Significado del acrónimo, del inglés *Si precediera del inglés*.

Introducción

1.1. Contexto y motivaciones

Los videojuegos constituyen hoy en día una de las industrias culturales y de entretenimiento más relevantes a nivel global. Los ingresos globales del sector alcanzaron los 188.800 millones de dólares en 2025, con una base de jugadores que supera los 3.600 millones de personas, lo que equivale al 61,5 % de la población mundial con acceso a internet [6]. Estas cifras sitúan a los videojuegos por encima de los ingresos combinados del *streaming* y la industria cinematográfica, consolidando su posición como el medio de entretenimiento dominante del siglo XXI.

Más allá de su peso económico, los videojuegos han demostrado tener un impacto positivo documentado en sus jugadores. Diversas investigaciones señalan que su uso moderado mejora capacidades cognitivas como la atención, la memoria de trabajo, la toma de decisiones y las habilidades espaciales [7]. Los juegos de estrategia, en particular, fomentan la autorregulación y la planificación, obligando al jugador a evaluar riesgos y anticipar movimientos bajo presión. Además, el componente interactivo y social de muchos títulos contribuye al desarrollo de valores como la cooperación y el trabajo en equipo [8]. El videojuego, por tanto, no es únicamente un producto de ocio, sino un medio expresivo con capacidad de influir de forma significativa en quienes lo experimentan.

Esta capacidad de generar experiencias entretenidas o memorables es precisamente lo que ha motivado el desarrollo del presente trabajo. A lo largo de los años, numerosos videojuegos han dejado una huella duradera: títulos que, independientemente de su presupuesto o escala, han conseguido transmitir emociones, plantear desafíos estimulantes o simplemente brindar momentos de disfrute genuino.

Poder contribuir, aunque sea a pequeña escala, a generar ese mismo tipo de experiencia en otras personas es la motivación principal de este proyecto. El desarrollo de un videojuego propio supone además una oportunidad única de adquirir de forma práctica las competencias técnicas y creativas que conforman esta industria: diseño de sistemas interactivos, implementación de lógica de juego, creación de interfaces y producción de contenido audiovisual.

1.2. Objetivos

El objetivo principal de este TFG es diseñar y desarrollar un videojuego por turnos completamente funcional utilizando el motor Godot y el lenguaje de programación C#, capaz de ofrecer una experiencia de juego completa y satisfactoria de principio a fin.

Para alcanzar este objetivo principal, se han definido los siguientes objetivos específicos:

- Diseñar e implementar la lógica fundamental del videojuego, en concreto, la estructura de un tablero, la definición de las piezas y las reglas de movimiento y combate entre ellas.
- Desarrollar un sistema de control de turnos y gestión del estado del juego que permita ejecutar partidas completas respetando las reglas definidas.
- Diseñar e implementar una interfaz de usuario clara e intuitiva que proporcione al jugador información visual sobre el estado del juego y permita interactuar con el sistema de manera sencilla.
- Incorporar elementos audiovisuales (animaciones y efectos sonoros) que mejoren la experiencia de juego y aporten mayor claridad a las acciones realizadas durante la partida.
- Adquirir experiencia práctica en el uso del motor de videojuegos Godot y en el desarrollo de sistemas interactivos con C#.

Como objetivo secundario, se plantea además:

- Implementar un bot inteligente capaz de analizar el estado del tablero y seleccionar los movimientos válidos más adecuados para cada situación, ofreciendo al jugador la posibilidad de disputar partidas contra un oponente desafiante.

1.3. Metodología

El desarrollo del TFG se ha llevado a cabo siguiendo el modelo de desarrollo **iterativo e incremental**, una metodología de construcción de software en la que el proyecto avanza a través de ciclos sucesivos denominados iteraciones, y en la que cada ciclo produce una versión del sistema ampliada y mejorada respecto a la anterior.

La distinción entre los términos *iterativo* e *incremental* es relevante. El componente *iterativo* hace referencia a la repetición sistemática de un conjunto de actividades (análisis, diseño, implementación y prueba) en cada ciclo de trabajo. El componente *incremental* implica que el resultado de cada iteración no es un prototipo desechable, sino un incremento funcional que se integra con el trabajo anterior y contribuye directamente al producto final. Ambas dimensiones se complementan, las iteraciones permiten incorporar aprendizajes y correcciones de forma temprana, mientras que los incrementos garantizan un avance continuo y medible hacia el sistema completo.

Frente al modelo en cascada, en el que cada fase debe completarse antes de iniciar la siguiente, el modelo iterativo e incremental entrelaza las actividades de especificación, diseño y desarrollo. Esto permite que los requisitos se vayan refinando a lo largo del proyecto a medida que se comprende mejor el problema y se evalúan los resultados de cada iteración [9]. Esta flexibilidad resulta especialmente adecuada para un proyecto como este, de carácter individual y con un alcance que ha ido precisándose de forma progresiva.

En la práctica, la adopción de esta metodología se ha concretado en una secuencia de entregas parciales en las que se han presentado versiones progresivas tanto del documento como del videojuego. Cada entrega ha sido revisada por los tutores del proyecto, cuyas observaciones han servido como retroalimentación para orientar la siguiente iteración. Este ciclo de trabajo, revisión y mejora ha permitido detectar de forma temprana errores conceptuales, problemas de diseño o decisiones técnicas poco adecuadas, evitando que dichos problemas se acumulen en fases posteriores. A su vez, cada iteración ha añadido nuevas funcionalidades o ha consolidado las ya existentes, de modo que el sistema ha crecido de forma coherente desde una versión inicial básica hasta el producto final completo y funcional.

1.4. Estructura del documento

Este documento se organiza en distintos capítulos, que dividen el proyecto en ...

Antecedentes y Estado del Arte

En este capítulo se analizan los antecedentes y el estado del arte relacionados con el TFG. Para ello, se realiza un recorrido por la evolución de los juegos de estrategia desde sus orígenes en los juegos de mesa clásicos hasta sus adaptaciones modernas, tanto físicas como digitales. Este análisis permite identificar las mecánicas fundamentales que han influido en el diseño del proyecto, así como contextualizar las decisiones adoptadas durante su desarrollo.

2.1. Evolución de los juegos de estrategia

Desde la antigüedad, los juegos de mesa han servido como una representación lúdica del conflicto, la planificación y la toma de decisiones. En las primeras civilizaciones, estos juegos cumplían una función tanto recreativa como cultural, y sentaron las bases de conceptos estratégicos que perduran hasta la actualidad.

Entre los ejemplos más antiguos se encuentra el *Juego Real de Ur*, originario de Iraq (aprox. del 2600-2400 a.C.) [10], presentaba un tablero dividido en secciones conectadas, tal y como se muestra en la Figura 2.1. Este diseño introducía decisiones tácticas relacionadas con el avance y la protección de las piezas, reforzando la importancia de la planificación a medio plazo. Ambos juegos, pese a su dependencia parcial del azar, contribuyeron a establecer una base conceptual sobre la que evolucionarían juegos de estrategia más complejos.

Aunque no se ha conservado un reglamento completo que permita reconstruir con total precisión las normas del juego, sí existe una descripción parcial recogida en una tablilla cuneiforme de origen babilonio, escrita entre los años 177 y 176 a.C. por el escriba Itti-Marduk-Balāṭu [1]. A partir de este testimonio y de los tableros conservados, se considera que el Juego Real de Ur consistía en una competición entre dos jugadores en forma de carrera a lo largo del tablero, con mecánicas comparables a juegos actuales como el parchís o el backgammon.

De forma similar, el juego egipcio *Senet*, documentado arqueológicamente durante el Imperio Nuevo, con ejemplares datados en el reinado de Amenhotep III (aprox. 1390–1353 a.C.) [2],

se desarrollaba sobre un tablero de treinta casillas, como se observa en la Figura 2.2. El desplazamiento de las piezas se determinaba mediante el lanzamiento de palos de conteo o huesos, introduciendo un componente de azar en la cantidad de casillas que podían avanzarse en cada turno. No obstante, el jugador conservaba un grado significativo de control estratégico, ya que podía decidir qué pieza mover, bloquear el avance del oponente o forzar retrocesos en determinadas situaciones. Esta combinación de aleatoriedad en la generación de movimientos y toma de decisiones tácticas en su aplicación convierte a *Senet* en uno de los primeros ejemplos conocidos de juego que equilibra azar y planificación. Aunque su mecánica era relativamente simple, introducía ya la idea de progresión de piezas y competición directa entre jugadores, aspectos clave en el desarrollo temprano del pensamiento estratégico [11].

Ambos juegos, pese a su dependencia parcial del azar, contribuyeron a establecer una base conceptual sobre la que evolucionarían juegos de estrategia más complejos.

Con el avance del tiempo, los juegos de estrategia comenzaron a abandonar progresivamente la dependencia del azar para centrarse en la planificación deliberada y la toma de decisiones informadas. En este contexto surge el *Ajedrez*, cuyo origen se sitúa en la India alrededor del siglo VI d.C., a partir del juego conocido como *Chaturanga* [12]. A diferencia de los juegos antiguos basados en carreras o desplazamientos condicionados por el azar, el ajedrez introduce una jerarquía claramente definida de piezas con capacidades diferenciadas y un sistema de captura directa. Estas características refuerzan conceptos como el sacrificio estratégico, el control del espacio y la planificación táctica a largo plazo. Asimismo, el ajedrez se caracteriza por ofrecer información completa a ambos jugadores, lo que lo convierte en un paradigma de la estrategia basada exclusivamente en la anticipación y el cálculo.

Dentro de esta evolución resulta especialmente relevante el juego chino *Dou Shou Qi*, también conocido como *Selva* o *Ajedrez animal*. A diferencia de juegos clásicos cuya antigüedad está bien documentada, el origen histórico de *Dou Shou Qi* no se encuentra claramente establecido. Las fuentes disponibles lo describen como un juego tradicional chino desarrollado con posterioridad al *Xiangqi*, del que probablemente deriva a nivel conceptual [3].

El juego se estructura en torno a una jerarquía asimétrica de unidades representadas por animales, cada uno con un rango definido, y emplea un sistema de captura por desplazamiento similar al de los juegos de ajedrez (Tablero en la Figura 2.3). No obstante, introduce excepciones explícitas en las reglas de captura —como la capacidad de una pieza de rango inferior para derrotar a otra superior bajo determinadas condiciones—, lo que rompe la jerarquía estricta y añade una dimensión estratégica basada en la gestión del riesgo y la anticipación del comportamiento del oponente. Este tipo de jerarquía no lineal anticipa principios fundamentales que aparecerán más adelante en los juegos de estrategia militar modernos.

El siguiente paso se produce a comienzos del siglo XX con la aparición de *L'Attaque*, un juego de mesa diseñado por Hermance Edan, quien registró su patente el 26 de noviembre de 1908 [13]. En su descripción original, el juego se define como un «*jeu de bataille avec des pièces mobiles sur damier*», es decir, un juego de batalla con piezas móviles sobre un tablero cuadriculado (Figura 2.4). La patente fue publicada oficialmente en 1909 por la Oficina de Patentes francesa, y el juego fue comercializado de forma continuada desde ese año hasta comienzos de la década de 1930.

L'Attaque introduce por primera vez de manera explícita la ocultación de información como mecánica central del diseño. Cada jugador dispone de un conjunto de piezas con valores numéricos jerárquicos, cuyo rango permanece oculto al oponente hasta el momento de la confrontación directa. El resultado del combate se resuelve comparando los valores de las piezas implicadas, eliminándose generalmente la de menor rango. La partida concluye cuando uno de los jugadores localiza y captura la pieza objetivo del adversario, identificada como la bandera.

Estas mecánicas suponen una ruptura clara con los juegos de información completa predominantes hasta ese momento, al trasladar una parte sustancial de la toma de decisiones al ámbito de la inferencia, el engaño y la gestión del riesgo. Asimismo, *L'Attaque* consolida una jerarquía militar explícita y permite la configuración libre de la disposición inicial de las unidades, lo que incrementa la profundidad estratégica desde el primer turno y refuerza la importancia de la planificación previa. En conjunto, estas innovaciones marcan un punto de inflexión en el diseño de juegos de estrategia, al introducir por primera vez de forma sistemática la incertidumbre como recurso estratégico.

La progresión histórica desde los primeros juegos de mesa hasta títulos como *L'Attaque* pone de manifiesto una evolución gradual desde mecánicas dominadas por el azar y el movimiento lineal hacia sistemas estratégicos cada vez más complejos, basados en la jerarquía de unidades, la confrontación directa y, finalmente, la ocultación de información. Esta acumulación progresiva de conceptos estratégicos, planificación, gestión del riesgo, engaño e inferencia, configura el marco teórico sobre el que se desarrollaría posteriormente *Stratego*.

2.2. El juego original: *Stratego*

El juego *Stratego* constituye uno de los referentes más influyentes dentro del ámbito de los juegos de estrategia con información imperfecta. Su diseño no surge de forma aislada, sino que se apoya en una serie de innovaciones mecánicas desarrolladas a comienzos del siglo XX, especialmente a partir del juego europeo *L'Attaque*, considerado su antecedente directo.

La versión moderna de *Stratego* fue desarrollada tras la Segunda Guerra Mundial por la compañía neerlandesa Jumbo, que adoptó una estética de inspiración militar napoleónica. El juego fue registrado comercialmente en 1960 y su difusión internacional se consolidó en 1961, cuando Milton Bradley obtuvo la licencia para su distribución en Estados Unidos [14].

Desde el punto de vista mecánico, *Stratego* se articula en torno a una jerarquía militar explícita de unidades, cuyos rangos permanecen ocultos al oponente hasta el momento del enfrentamiento directo. El sistema de combate se resuelve mediante la comparación de los valores de las piezas implicadas, eliminándose generalmente la de menor rango. El objetivo principal de la partida consiste en localizar y capturar la pieza objetivo del adversario, identificada como la bandera.

Una de las decisiones de diseño más relevantes es la configuración inicial libre del despliegue de las unidades, que traslada una parte sustancial del peso estratégico al momento previo al inicio de la partida. Esta característica, combinada con la ocultación de información, fomenta el engaño, la deducción y la gestión del riesgo como elementos centrales de la experiencia de juego.

Desde el punto de vista material, *Stratego* ha experimentado diversas adaptaciones físicas a lo largo del tiempo. Las primeras ediciones utilizaban piezas de cartón impreso, que fueron sustituidas posteriormente por piezas de madera pintada. A partir de finales de la década de 1960, estas fueron reemplazadas por piezas de plástico, más económicas y estables. Este cambio no solo respondió a criterios de producción, sino que también redujo el riesgo de vuelco accidental de las piezas, un problema crítico en un juego donde la revelación involuntaria de la información compromete el desarrollo estratégico de la partida.

Desde una perspectiva de diseño, *Stratego* se caracteriza por la ausencia total de elementos aleatorios durante la partida. Todas las decisiones se basan en la planificación, la inferencia y la lectura del comportamiento del oponente, lo que lo convierte en un paradigma de la estrategia bajo información imperfecta.

2.3. Variaciones, adaptaciones y videojuegos relacionados

Además de sus múltiples ediciones físicas, *Stratego* ha dado lugar a numerosas variantes temáticas ambientadas en universos de ficción como *Star Wars*, *El Señor de los Anillos*, o *Marvel*. Estas adaptaciones mantienen la estructura básica del juego original, introduciendo cambios estéticos y ajustes menores en las reglas según la ambientación.

En el ámbito digital, *Stratego* cuenta con versiones en línea que permiten partidas a distancia entre jugadores de todo el mundo como el *Stratego Online*¹ y apps móviles disponibles en iOS y Android. Estas plataformas incorporan modos clásicos y variantes con tableros alternativos o un número reducido de piezas, contribuyendo a mantener vigente el interés por el juego. El juego conserva además presencia activa en el ámbito competitivo, especialmente en países como Países Bajos, Alemania y Bélgica, incluyendo torneos oficiales y rankings europeos.

Junto a *Stratego*, existen otros juegos de mesa y digitales basados en principios similares. Un ejemplo es *Luzhanqi* [5], originario de China, cuyo tablero se muestra en la Figura 2.5. Este juego comparte la mecánica de piezas ocultas y configuración libre inicial, evidenciando una convergencia de ideas estratégicas entre distintas tradiciones culturales.

En el ámbito de los videojuegos, destacan títulos de estrategia por turnos que enfatizan la planificación, la jerarquía de unidades y la toma de decisiones en entornos discretos. Entre ellos se encuentran la saga *Advance Wars* (Intelligent Systems, Advance Wars series, Nintendo) y *Into the Breach* (Subset Games, Into the Breach). En ambos casos, aunque la información sobre el enemigo es visible, el peso estratégico recae en la gestión de unidades, el posicionamiento y la anticipación de las acciones del rival.

Dentro del ámbito de los videojuegos de estrategia por turnos que comparten principios mecánicos con *Stratego*, destacan títulos como la saga *Advance Wars*. Esta serie, desarrollada por Intelligent Systems para plataformas de Nintendo entre 2001 y 2019, se basa en combates tácticos sobre un tablero cuadriculado donde los jugadores despliegan y gestionan unidades con jerarquías y características diferenciadas. Cada unidad posee fortalezas y debilidades específicas frente a

¹<https://www.stratego.com>

otros tipos de unidades, lo que obliga a planificar los movimientos cuidadosamente y anticipar las acciones del adversario. Aunque la información sobre la ubicación de las unidades enemigas es visible, la profundidad estratégica recae en la gestión eficiente de recursos, el posicionamiento táctico y la secuencia de ataques, elementos que recuerdan la planificación y la jerarquía presentes en *Stratego*.

De manera similar, el videojuego *Into the Breach* (Subset Games, 2018) traslada el concepto de estrategia por turnos a un entorno de tablero discretizado donde cada unidad tiene capacidades específicas y limitadas. La mecánica principal consiste en anticipar el comportamiento del enemigo y planificar acciones para proteger objetivos clave, equilibrando riesgo y recompensa en cada turno. Esta combinación de toma de decisiones estrictamente calculada, jerarquía de unidades y confrontación directa recuerda la esencia de *Stratego*, donde el control del espacio y la gestión de las piezas son determinantes para la victoria. La simplicidad aparente del tablero contrasta con la profundidad estratégica requerida, consolidando a *Into the Breach* como un ejemplo contemporáneo de cómo los principios clásicos de los juegos de estrategia pueden adaptarse eficazmente al medio digital.

Estas aproximaciones digitales ponen de manifiesto la diversidad de soluciones adoptadas en el diseño de videojuegos de estrategia y sirven como referencia para el desarrollo del TFG. El objetivo del proyecto es combinar la jerarquía de unidades y la confrontación directa propias de *Stratego* con las ventajas del medio digital, manteniendo la tensión estratégica derivada de la información oculta y aprovechando las posibilidades de automatización, accesibilidad y escalabilidad que ofrece el formato de videojuego.

2.4. Motores de desarrollo de videojuegos

Un motor de videojuegos es un *framework* de desarrollo que proporciona un conjunto integrado de herramientas, librerías y sistemas sobre los que construir un videojuego. Estos motores abstraen gran parte de la complejidad técnica inherente al desarrollo, ofreciendo soluciones ya implementadas para tareas como el renderizado gráfico, la gestión de escenas, el control de entrada del usuario, la simulación de físicas o la exportación a distintas plataformas. Gracias a esta abstracción, el desarrollador puede centrarse principalmente en la lógica del juego, el diseño de mecánicas y la experiencia del usuario.

En la actualidad, el panorama del desarrollo de videojuegos independiente está dominado por diversos motores de videojuegos, entre los que actualmente destacan más tenemos **Unreal Engine**, **Unity** y **Godot**. Los tres cuentan con una amplia presencia tanto en el ámbito profesional como en el educativo, y representan enfoques distintos ante el desarrollo.

Unreal Engine, desarrollado por Epic Games, es el motor de referencia en producciones de gran presupuesto y alta exigencia gráfica. Sus sistemas avanzados de renderizado, iluminación y simulación física permiten desarrollar entornos tridimensionales de alta fidelidad visual, lo que lo convierte en la herramienta estándar en la industria AAA. Esta potencia técnica tiene como contrapartida una curva de aprendizaje pronunciada, unos requisitos de hardware elevados

y tiempos de compilación considerables, factores que pueden suponer una desventaja significativa en proyectos de pequeña escala o con recursos limitados [15].

Unity se ha consolidado como uno de los motores más populares tanto en el ámbito independiente como en el educativo, gracias a su versatilidad para desarrollar videojuegos en dos y tres dimensiones sobre una amplia variedad de plataformas. Cuenta con una comunidad muy extensa, una gran cantidad de documentación y un ecosistema amplio de herramientas y recursos. Sin embargo, su arquitectura interna está históricamente orientada al desarrollo tridimensional, lo que puede introducir una sobrecarga innecesaria en proyectos 2D más sencillos. Además, la controversia generada en 2023 [16, 17] por una propuesta de tarificación por instalación del juego, que fue ampliamente rechazada por la comunidad de desarrolladores y finalmente revertida, puso en evidencia los riesgos asociados a depender de un motor de código cerrado con modelo de licencia comercial [18].

Godot es un motor de código abierto con licencia MIT que ha experimentado un crecimiento notable en los últimos años, especialmente dentro del desarrollo independiente. Su filosofía se basa en la simplicidad, la modularidad y la accesibilidad. A diferencia de Unity, su motor 2D no es una adaptación del motor 3D sino un sistema nativo e independiente, lo que lo convierte en una herramienta especialmente eficiente para el desarrollo de juegos bidimensionales. Su editor ocupa menos de 200 MB, arranca en segundos y permite ciclos de iteración muy rápidos, aspectos especialmente valiosos en proyectos individuales. A esto se suma la ausencia total de costes de licencia y la imposibilidad estructural de que las condiciones de uso cambien de forma unilateral, al tratarse de software de código abierto [19].

La percepción de Godot como un motor de menor capacidad frente a sus competidores ha quedado desmentida de forma contundente con el lanzamiento de *Slay the Spire II* (MegaCrit, 2026). El estudio, que había iniciado el desarrollo del juego en Unity, migró el proyecto íntegramente a Godot tras la polémica de 2023, asumiendo reescribir más de dos años de trabajo [20]. El resultado ha sido uno de los lanzamientos independientes más exitosos de los últimos años, en sus primeros 3 días en Early Access alcanzó un pico de más de 500.000 jugadores simultáneos en Steam, y en su primera semana superó los tres millones de unidades vendidas [21]. Este caso demuestra que Godot es capaz de sustentar productos comerciales de alto nivel cuando el tipo de juego se alinea con sus fortalezas.

La Tabla 2.1 resume las principales diferencias entre los tres motores considerando los criterios más relevantes para el contexto de este proyecto.

Criterio	Unreal Engine	Unity	Godot
Rendimiento en 3D	Muy alto	Alto	Medio
Rendimiento en 2D	Medio	Alto	Muy alto
Curva de aprendizaje	Alta	Media	Baja
Requisitos de hardware	Altos	Medios	Bajos
Modelo de licencia	Comercial	Comercial	Código abierto
Adecuación a proyectos individuales	Media	Alta	Muy alta
Ecosistema y comunidad	Muy amplio	Muy amplio	En crecimiento

Tabla 2.1: Comparativa entre motores de desarrollo de videojuegos

Tras este análisis, Godot se presenta como el motor más adecuado para el desarrollo de este TFG. Al tratarse de un videojuego de estrategia por turnos en dos dimensiones, con un alcance controlado y desarrollado en un contexto académico individual, las ventajas de Godot en términos de rendimiento nativo en 2D, ligereza, rapidez de iteración y ausencia de costes de licencia superan con claridad sus limitaciones en otros ámbitos. Su menor ecosistema de extensiones no supone ninguna restricción para un proyecto de esta escala, y su modelo de código abierto ofrece una estabilidad y previsibilidad que los motores comerciales no pueden garantizar.

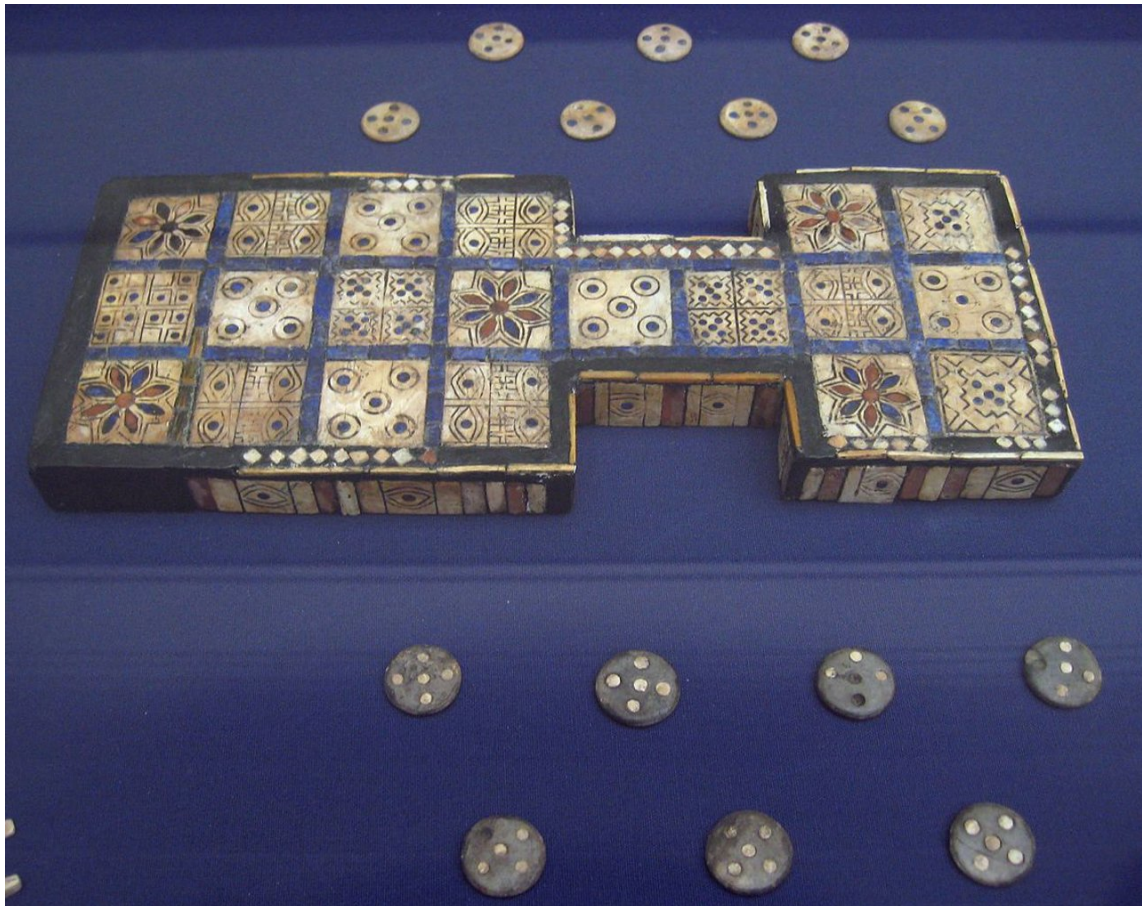


Figura 2.1: Tablero del Juego Real de Ur [1]



Figura 2.2: Tablero del juego Senet [2]

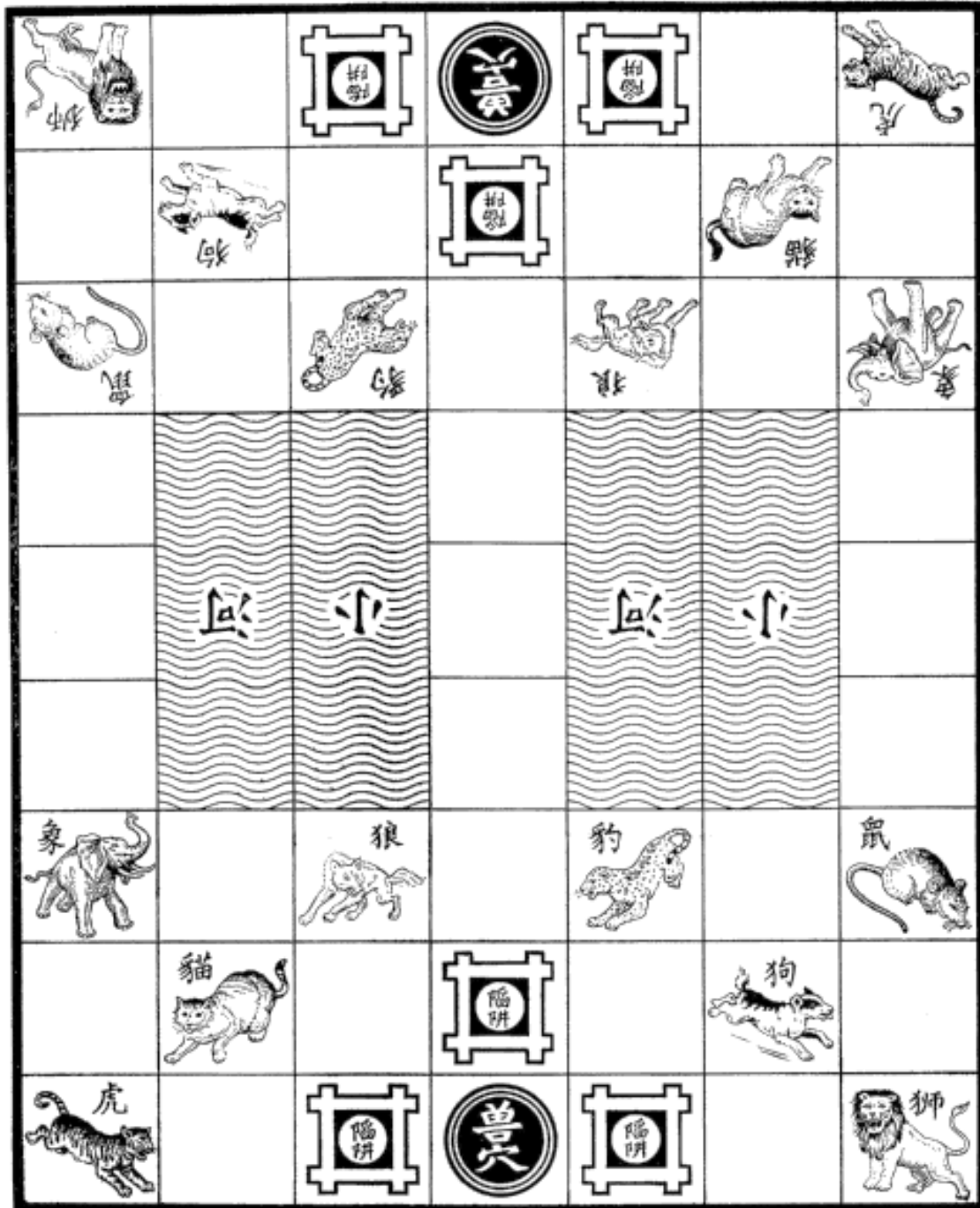


Figura 2.3: Tablero del juego Dou Shou Qi [3]

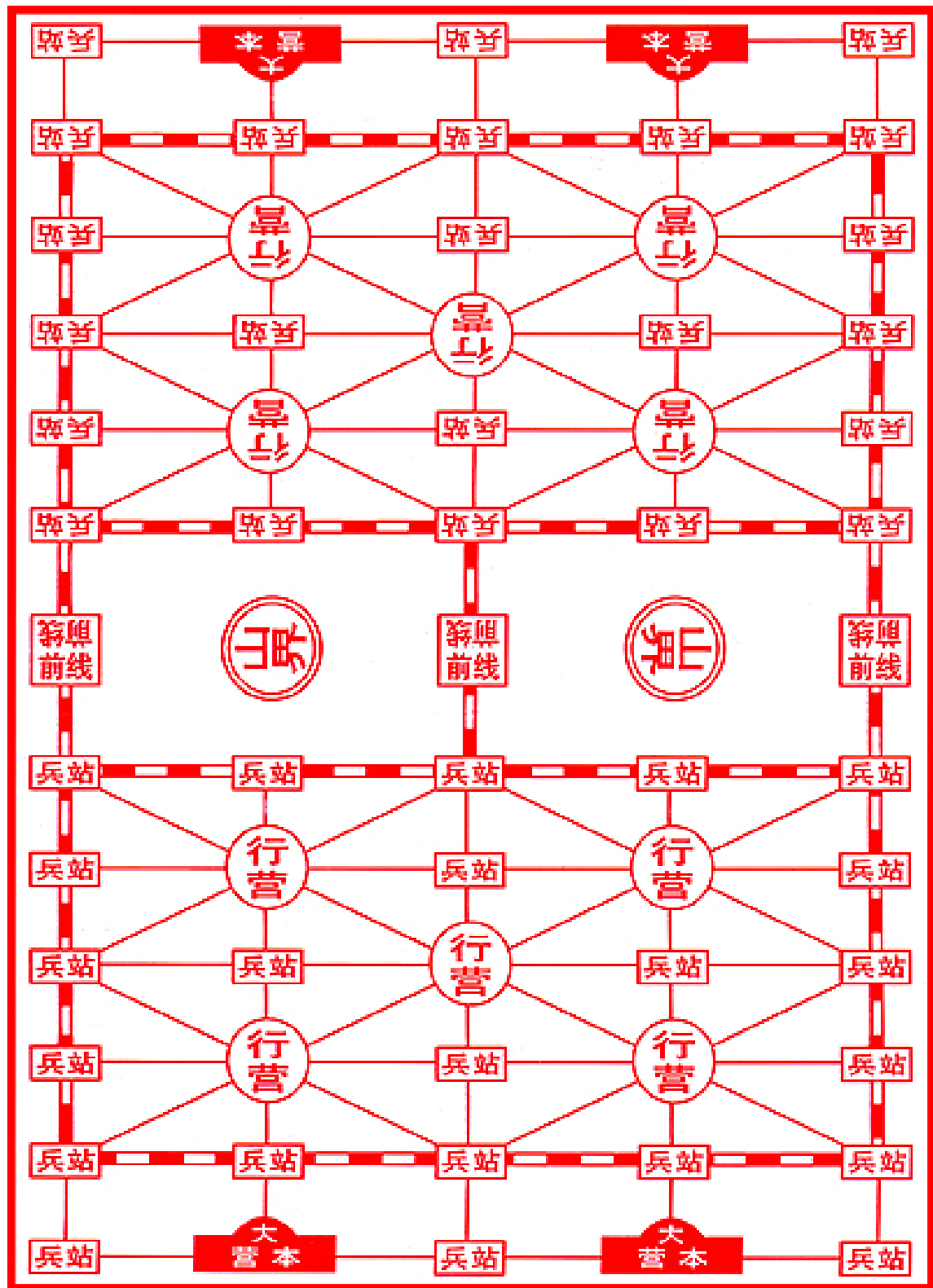


Figura 2.5: Tablero del juego Luzhanqi [5]

Concepto del videojuego

En este capítulo se describe el diseño general de *NeuroForge*, abordando sus mecánicas principales, reglas de juego y los elementos que definen su experiencia estratégica. Se detallan el funcionamiento del sistema de juego, la interacción con el jugador, la ambientación y el público al que va dirigido.

3.1. Resumen del juego

NeuroForge es un videojuego de estrategia por turnos para un jugador inspirado en el clásico *Stratego*. Dos bandos se enfrentan en un tablero de 10×10 casillas: el jugador humano y un oponente controlado por la máquina. El objetivo es destruir el **núcleo de energía** rival o dejarlo sin movimientos posibles.

Cada bando despliega en secreto un conjunto de piezas de distintos rangos en las cuatro filas más cercanas a su borde del tablero. Las identidades permanecen ocultas para el rival hasta que dos piezas entran en combate, momento en el que ambas se revelan y se resuelve el enfrentamiento según su rango.

En cada turno, el jugador puede realizar una única acción: mover una pieza a una casilla adyacente vacía, o moverla a una casilla ocupada por una pieza enemiga para iniciar un combate. Existe una excepción, los **exploradores**, que pueden desplazarse varias casillas en línea recta en un mismo turno. El combate se resuelve de la siguiente forma:

- La pieza de mayor rango vence y ocupa la casilla de la derrotada.
- Si ambas piezas tienen el mismo rango, ambas son eliminadas del tablero.
- Ciertas piezas tienen reglas especiales que se detallan más adelante.

3.2. Mecánicas

Gran parte del atractivo de *NeuroForge* reside en el desconocimiento mutuo de la identidad de las piezas rivales. Esto convierte la deducción y el engaño en elementos estratégicos centrales: el jugador debe inferir la disposición del enemigo a partir de su comportamiento, al tiempo que protege la información sobre sus propias piezas.

3.2.1. Tablero

El tablero es una cuadrícula de 10×10 casillas (Figura 3.1). Las cuatro filas más próximas a cada borde constituyen el **área de despliegue** de cada jugador, donde se colocan las piezas al inicio de la partida. Las dos filas centrales permanecen despejadas al comienzo y son terreno neutral.

En el centro del tablero se encuentran **casillas intransitables**, dispuestas en dos zonas de 2×2 . Ninguna pieza puede moverse a ellas, ocuparlas ni atravesarlas. Estos obstáculos dividen el tablero en dos mitades y condicionan las rutas de ataque y defensa, actuando como cuellos de botella que añaden profundidad táctica al posicionamiento.

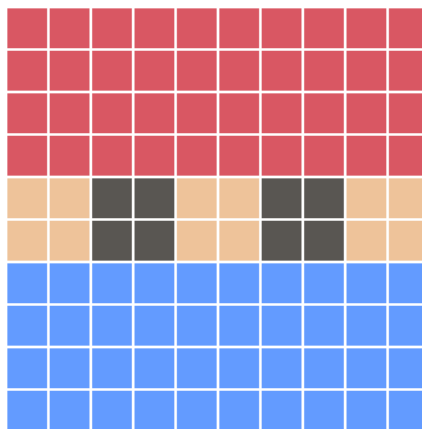


Figura 3.1: Esquema del tablero

3.2.2. Colocación inicial

Al inicio de cada partida, ambos bandos colocan sus piezas dentro de su respectiva área de despliegue con la distribución que consideren oportuna. No es posible colocar piezas fuera de dicha área antes de comenzar la partida. Una vez confirmada la colocación, la partida da comienzo y ningún jugador conoce la identidad de las piezas rivales.

La fase de despliegue tiene un peso estratégico considerable: una disposición bien planificada puede proteger las piezas clave, dificultar la deducción del rival y preparar aperturas ofensivas o defensivas para los primeros turnos.

3.2.3. Piezas

Las piezas se rigen por un sistema de rangos que determina el resultado de los combates: la pieza de mayor rango vence a la de menor rango. Cada tipo de pieza tiene una cantidad limitada por bando. Algunas piezas cuentan con propiedades especiales que modifican las reglas generales de movimiento o combate. A continuación se describen todos los tipos disponibles:

- **Núcleo de energía (cantidad: 1):** La pieza más importante del bando. No puede moverse. Su destrucción supone la derrota inmediata de su propietario, y puede ser atacado por cualquier pieza enemiga capaz de moverse.
- **Torretas automáticas (cantidad: 6):** Estructuras defensivas inmóviles que no pueden atacar, pero destruyen automáticamente cualquier pieza que las ataque, independientemente de su rango. La única excepción son los **saboteadores**, que son los únicos capaces de destruirlas.
- **Máquina de guerra (cantidad: 1, rango 10):** La unidad de combate más poderosa. Vence a cualquier pieza por rango, salvo al **Phantom** si este ataca primero.
- **Nova (cantidad: 1, rango 9):** Sin propiedades especiales.
- **Mecha (cantidad: 2, rango 8):** Sin propiedades especiales.
- **Centinela (cantidad: 3, rango 7):** Sin propiedades especiales.
- **Canino (cantidad: 4, rango 6):** Sin propiedades especiales.
- **Cyborg (cantidad: 4, rango 5):** Sin propiedades especiales.
- **Soldado (cantidad: 4, rango 4):** Sin propiedades especiales.
- **Saboteador (cantidad: 5, rango 3):** La única pieza capaz de destruir las **torretas automáticas**. En cualquier otro combate se rige por las reglas generales de rango.
- **Explorador (cantidad: 6, rango 2):** Puede desplazarse cualquier número de casillas en línea recta, horizontal o verticalmente, siempre que no haya obstáculos ni piezas en su camino. Si se desplaza más de una casilla en un turno, **su identidad queda revelada**.
- **Phantom (cantidad: 1, rango 1):** La pieza de menor rango. Su única ventaja es que derrota a la **Máquina de guerra** si es quien inicia el ataque. En cualquier otro combate pierde por rango.

3.2.4. Turnos

La partida se desarrolla en turnos alternos, comenzando siempre el jugador humano. En cada turno se realiza una única acción con una sola pieza: moverla a una casilla vacía adyacente, o moverla a una casilla ocupada por una pieza enemiga para iniciar un combate. No es posible mover y atacar en el mismo turno, salvo en el caso del **explorador**, que puede cubrir varias

casillas en un único movimiento y atacar al final de su recorrido si la casilla de destino está ocupada por un enemigo.

Las reglas que rigen el movimiento son las siguientes. Solo se puede mover una pieza por turno, desplazándola una única casilla en cualquiera de las cuatro direcciones ortogonales (arriba, abajo, izquierda, derecha), nunca en diagonal. No puede haber dos piezas en la misma casilla, y ninguna pieza puede saltar ni atravesar casillas ocupadas por otras piezas o por zonas intransitables. Las piezas sin capacidad de movimiento, como el **núcleo de energía** y las **torretas automáticas**, permanecen fijas durante toda la partida.

Para evitar situaciones de pasividad o movimientos repetitivos sin intención estratégica, existe una restricción adicional, una pieza no puede moverse entre 2 casillas durante **mas de tres turnos**, al cuarto turno se le bloqueara la posibilidad de regresar a la casilla de la que viene hasta el siguiente turno o se mueva a otra distinta. Esta limitación afecta únicamente a la pieza que salió de esa casilla, otras piezas pueden ocuparla sin restricción.

En cuanto al ataque, este se produce cuando una pieza se desplaza a una casilla ocupada por una pieza enemiga, iniciando así un combate. Si la pieza atacante vence, ocupa la casilla de la pieza destruida. Si pierde, es ella quien es eliminada y la pieza defendida permanece en su posición.

Si al comienzo del turno de un bando ninguna de sus piezas puede realizar movimiento o ataque válido alguno, la partida termina y ese bando pierde.

3.2.5. Resolución de combates

El combate se produce cuando una pieza se desplaza a una casilla ocupada por una pieza rival. En ese momento, la identidad de ambas piezas queda revelada y el resultado se determina según las siguientes reglas:

- Si la pieza atacante tiene **mayor rango**, vence: la pieza rival es destruida y el atacante ocupa su casilla.
- Si la pieza atacante tiene **menor rango**, es destruida y la pieza defensora permanece en su posición.
- Si ambas piezas tienen el **mismo rango**, el combate termina en empate y ambas son eliminadas del tablero.
- Una **torreta automática** destruye a cualquier pieza que la ataque, excepto al **saboteador**, que la destruye y ocupa su casilla.
- Si el **Phantom** ataca a la **Máquina de guerra**, el Phantom vence independientemente del rango.

3.2.6. Condiciones de victoria

La partida puede concluir de dos formas. La primera es la destrucción del **núcleo de energía** rival, lo que supone la derrota inmediata de su propietario.

La segunda es que uno de los bandos quede sin movimientos posibles, es decir, si todas sus piezas con capacidad de moverse han sido destruidas o bloqueadas, ese bando pierde. En ambos casos, el bando que provoca la condición de victoria gana la partida.

3.2.7. Interacción y control

El jugador realiza todas sus acciones mediante el ratón. El sistema de interacción está diseñado para que las posibilidades disponibles en cada momento sean siempre visualmente evidentes, minimizando la necesidad de memorizar reglas durante el juego.

Al inicio del turno del jugador, las piezas que pueden actuar aparecen resaltadas en el tablero; las que no tienen movimientos disponibles no se destacan. Para mover una pieza, cuando el jugador pulsa sobre ella, lo que hace que se muestren las casillas de destino válidas. Haciendo click en una de esas casillas, el movimiento queda confirmado. Si se hace click en cualquier otro lado que no sea una de esas casillas (incluyendo la propia pieza), la acción se cancela y el jugador puede volver a intentarlo.

Al desplazarse a una casilla vacía, el turno finaliza y pasa al oponente. Al desplazarse a una casilla ocupada por una pieza enemiga, se inicia el combate, ambas piezas se resaltan brevemente en pantalla, las piezas ocultas se revelan si aún no habían sido identificadas, y el resultado se resuelve según las reglas establecidas. La pieza perdedora desaparece del tablero y la ganadora queda visible en la casilla donde se produjo el enfrentamiento. Cada acción se acompaña de animaciones y efectos sonoros que refuerzan la claridad de lo ocurrido.

Una vez concluida la acción del jugador, el bot ejecuta su turno de forma automática. El jugador observa el movimiento del oponente y recupera el control al inicio de su siguiente turno.

3.3. Ambientación

NeuroForge sitúa al jugador en un universo de ciencia ficción futurista en el que las guerras se libran mediante ejércitos de inteligencias artificiales, drones y armamento avanzado. El título hace referencia al proceso de forjar una red neuronal, cada partida representa un ciclo de entrenamiento intensivo en el que una IA militar es sometida a prueba, refinada y preparada para el combate real.

Los enfrentamientos no tienen lugar en un campo de batalla físico, sino dentro de un **programa de simulación de combate de alta fidelidad**, desarrollado para evaluar y optimizar las capacidades estratégicas de estas inteligencias antes de su despliegue. Cada simulación es un escenario controlado en el que las unidades son instancias virtuales, los combates son pruebas de

rendimiento y cada resultado, victoria o derrota, alimenta el proceso de entrenamiento de la IA, ajustando sus parámetros para el siguiente ciclo.

Unidades y estructuras

- **Núcleo de energía:** Centro vital de cada bando. Su destrucción interrumpe el suministro energético de todas las unidades y supone la derrota inmediata.
- **Torretas automáticas:** Sistemas defensivos estáticos con capacidad de eliminar cualquier amenaza, salvo los **saboteadores**, que las neutralizan mediante camuflaje avanzado.
- **Máquina de guerra:** Mecha masivo pilotado por la IA suprema del bando. La unidad más poderosa del campo de batalla, vulnerable únicamente frente al ataque de malware del **Phantom**.
- **Nova:** Guardia cyborg de alto rango con partes orgánicas prácticamente inexistentes. Coordina el despliegue de las fuerzas en combate.
- **Mecha:** Robots de combate pesados diseñados para resistir y dominar en enfrentamientos directos.
- **Centinela:** Unidades de asalto mecanizadas optimizadas para el combate, con una resistencia estructural muy superior a cualquier unidad humana.
- **Canino:** Cuadrúpedos mecánicos de alta movilidad diseñados para la interceptación rápida.
- **Cyborg:** Soldados rápidos y eficaces con implantes cibernéticos ligeros.
- **Soldado:** Refuerzos humanos destinados a mantener las líneas de combate. La unidad estándar de menor rango.
- **Saboteador:** Especialistas en infiltración capaces de neutralizar torretas mediante tecnología de camuflaje avanzado. Indefensos en combate directo contra otras unidades.
- **Explorador:** Drones de reconocimiento ágiles capaces de recorrer largas distancias en línea recta. Si se desplazan más de una casilla en un turno, su identidad queda revelada, ya que son los únicos con ese tipo de movimiento.
- **Phantom:** Dron de infiltración diseñado con un único propósito: destruir a la **Máquina de guerra** enemiga mediante malware especializado. Es la unidad más débil en un combate convencional, pero puede destruir instantáneamente a la **Máquina de guerra** si ataca primero.

Escenario

Los escenarios de entrenamiento se generan a partir de registros digitalizados de antiguos campos de batalla. Las zonas intransitables del tablero representan sectores de interferencia

energética crítica: áreas contaminadas por los residuos de armamento avanzado de conflictos anteriores, cuyos datos han sido incorporados al entorno simulado como obstáculos permanentes. Dentro de estos sectores, las descargas residuales y las interferencias electromagnéticas imposibilitan el funcionamiento de cualquier unidad, impidiendo su entrada o tránsito. En la simulación, como en la realidad, estas zonas solo se vuelven transitables con el tiempo, una vez que la contaminación energética se disipa de forma natural.

3.4. Diseño de niveles

NeuroForge se desarrolla sobre un único tablero estándar de 10×10 casillas. Pese a su aparente simplicidad, las zonas intransitables del centro generan cuellos de botella que condicionan las rutas de movimiento y obligan a planificar cuidadosamente las aperturas y las líneas de avance.

El diseño actual se centra en este tablero como escenario principal. No obstante, la arquitectura modular del juego abre la puerta a posibles expansiones futuras, entre las que podrían contemplarse tableros de dimensiones distintas, eventos ambientales que alteren temporalmente la movilidad de ciertas piezas, o casillas especiales con efectos estratégicos concretos. Estas posibilidades no forman parte del alcance del proyecto, pero ilustran la escalabilidad del diseño sin necesidad de modificar las mecánicas fundamentales.

3.5. Audiencia

NeuroForge está orientado a jugadores con interés en la estrategia por turnos, tanto quienes tienen experiencia previa en el género como quienes quieran adentrarse en él por primera vez. El perfil del público objetivo incluye a personas con afinidad por juegos de lógica, planificación y táctica, como el ajedrez o títulos clásicos de estrategia, con independencia de su edad.

El diseño prioriza la claridad en las mecánicas y la profundidad en la toma de decisiones, buscando que el juego sea accesible sin sacrificar la complejidad estratégica. Aunque el estilo visual y la ambientación pueden resultar especialmente atractivos para un público joven y adulto, las reglas no presentan ninguna barrera de entrada significativa para jugadores de cualquier franja de edad.

En cuanto a la clasificación por edades, el contenido del juego no incluye violencia explícita, lenguaje ofensivo ni temáticas sensibles. Atendiendo a los criterios del sistema europeo *Pan European Game Information* (PEGI), el videojuego podría encuadrarse en una clasificación de **PEGI 3** o **PEGI 7**, al tratarse de una experiencia estratégica abstracta sin representaciones realistas de violencia.

Planificación

En este capítulo se presenta la planificación del desarrollo del proyecto *NeuroForge*, definiendo la organización temporal del trabajo, los recursos empleados, las restricciones existentes y los posibles riesgos identificados.

La planificación se ha concebido como una guía flexible, susceptible de ser ajustada a lo largo del desarrollo conforme se adquiría una visión más precisa del alcance real del proyecto y de las dificultades técnicas asociadas a su implementación.

4.1. Planificación inicial

La planificación inicial del proyecto se realizó con el objetivo de establecer una estimación razonable del trabajo a desarrollar, sirviendo como referencia para organizar las tareas y distribuir el esfuerzo a lo largo del periodo de realización del TFG. Esta planificación no se concibe como un plan rígido, sino como una guía orientativa, susceptible de ajustes conforme avanzaba el desarrollo del proyecto.

El desarrollo del videojuego se planteó siguiendo una metodología iterativa e incremental, de modo que cada iteración se centra en un conjunto concreto de objetivos y funcionalidades, permitiendo validar progresivamente el estado del sistema y añadir nuevas capacidades sobre una base previamente funcional.

4.1.1. Iteración 1: Implementación del juego base

El objetivo de esta primera iteración es desarrollar una versión funcional mínima del videojuego, sin elementos visuales pulidos ni inteligencia artificial, que sirva como base estructural para el resto del proyecto. Se estima una dedicación de **20 horas** para el conjunto de tareas de esta fase.

- **Tablero y casillas** (2 h): Implementación de la cuadrícula de 10×10 casillas, incluyendo las zonas intransitables del centro del tablero.

- **Definición de piezas** (3 h): Desarrollo de los distintos tipos de piezas con sus propiedades individuales: rangos, piezas inmóviles, y reglas especiales de combate y movimiento.
- **Sistema de movimiento** (3 h): Implementación de la selección de piezas, la visualización de las casillas de destino válidas y el desplazamiento sobre una casilla válida.
- **Sistema de combate** (3 h): Resolución inmediata de los enfrentamientos entre piezas según las reglas de rango definidas, sin animaciones ni retroalimentación visual elaborada.
- **Sistema de ocultación de piezas** (2 h): Implementación de la mecánica de identidades ocultas, de forma que las piezas rivales no muestran su tipo hasta que entran en combate.
- **Despliegue inicial aleatorio del bot** (1 h): Colocación automática de las piezas del oponente en su zona de despliegue de forma aleatoria al comienzo de la partida.
- **Bot de movimiento aleatorio** (1 h): Implementación de un oponente básico que ejecuta movimientos válidos de forma aleatoria durante su turno.
- **Sistema de despliegue del jugador** (4 h): Pantalla básica que permite al jugador colocar sus piezas en su zona antes de comenzar la partida.
- **Condición de fin de partida** (1 h): Detección de las condiciones de victoria y derrota, con parada completa del juego al producirse alguna de ellas.

4.1.2. Iteración 2: Interfaz de usuario y pulido visual

El objetivo de esta iteración es dotar al juego de una interfaz completa, visualmente cuidada e intuitiva, sustituyendo los elementos provisionales de la iteración anterior. Por su volumen y variedad de elementos, esta iteración se planificó como la más extensa de las dedicadas a desarrollo visual, con una estimación de **56 horas**.

- **Bocetado de interfaces** (8 h): Elaboración de bocetos y propuestas visuales para todas las pantallas del juego antes de su implementación, con el fin de planificar la disposición de los elementos y la navegación entre escenas.
- **Búsqueda e integración de assets** (8 h): Selección de recursos gráficos para construir las interfaces de forma visualmente coherente con la ambientación del juego.
- **Animaciones de movimiento de piezas** (1 h): Incorporación de animaciones básicas al desplazamiento de las piezas sobre el tablero, mejorando la claridad de las acciones realizadas.
- **Rediseño visual del tablero** (4 h): Mejora de la representación gráfica del tablero y de los indicadores de selección y movimiento disponible, incluyendo animaciones asociadas.
- **Sprites de las piezas** (10 h): Creación e integración de la representación gráfica de cada tipo de pieza.
- **Interfaz de despliegue mejorada** (4 h): Rediseño de la pantalla de colocación inicial de piezas, haciéndola más clara e intuitiva para el jugador.

- **Interfaz de resolución de combate** (6 h): Pantalla que muestra el resultado de cada enfrentamiento, indicando la pieza ganadora y perdedora y los motivos del resultado según las reglas aplicadas.
- **Pantalla de fin de partida** (1 h): Escena que informa al jugador del resultado final y le permite iniciar una nueva partida o volver al menú principal.
- **Menú principal** (1 h): Escena inicial con opciones para comenzar una partida, consultar las reglas y salir del juego.
- **Menú de pausa** (2 h): Escena accesible durante la partida que permite al jugador continuar, consultar las reglas o salir al menú principal.
- **Menú de reglas** (8 h): Escena con múltiples viñetas que explican de forma visual y progresiva todas las mecánicas y reglas del juego.
- **Ajustes y correcciones** (3 h): Revisión general de los elementos implementados en la iteración anterior y aplicación de los cambios que no pudieron contemplarse durante su desarrollo.

4.1.3. Iteración 3: Mejoras audiovisuales

El objetivo de esta iteración es completar la experiencia audiovisual del juego mediante la incorporación de sonido y los últimos elementos visuales de ambientación. Se estima una dedicación de **15 horas** para el conjunto de tareas de esta fase.

- **Fondos y elementos decorativos** (2 h): Incorporación de fondos y elementos visuales de carácter estético que complementan la ambientación del juego sin afectar a la jugabilidad.
- **Efectos de sonido** (6 h): Integración de efectos sonoros asociados a las distintas acciones del juego, como movimientos, combates y navegación por menús.
- **Música** (3 h): Incorporación de música de fondo para las distintas escenas del juego, reforzando la ambientación.
- **Menú de opciones de audio** (4 h): Implementación de una pantalla de configuración que permite al jugador ajustar de forma independiente el volumen de los efectos de sonido y de la música.

4.1.4. Iteración 4: Implementación del bot inteligente

El objetivo de esta iteración es sustituir el bot de movimiento aleatorio por un sistema de inteligencia artificial capaz de tomar decisiones estratégicas durante la partida. Por la complejidad técnica que implica, esta iteración se planificó como la más extensa del proyecto, con una estimación de **55 horas**.

- **Diseño del modelo de IA** (16 h): Definición del enfoque y la arquitectura del sistema de toma de decisiones del bot, teniendo en cuenta las particularidades del juego, como la información oculta de las piezas rivales.
- **Entrenamiento del modelo** (25 h): Proceso de entrenamiento de la IA mediante partidas simuladas, con el fin de ajustar su comportamiento y mejorar progresivamente la calidad de sus decisiones.
- **Integración del bot inteligente** (6 h): Sustitución del bot aleatorio existente por el modelo entrenado, asegurando su correcta integración con el sistema de turnos y las reglas del juego.
- **Evaluación del comportamiento** (8 h): Pruebas del bot en distintas situaciones de partida para verificar que sus decisiones son coherentes con las reglas y que no produce errores en la ejecución.

4.1.5. Iteración 5: Pulido final

El objetivo de esta última iteración es revisar el conjunto del sistema, corregir los errores detectados y garantizar la estabilidad y calidad de la versión final del videojuego. Se estima una dedicación de **14 horas** para el conjunto de tareas de esta fase.

- **Optimización del rendimiento** (4 h): Análisis y mejora del rendimiento del sistema en caso de detectarse problemas de fluidez o consumo de recursos.
- **Revisión del feedback al jugador** (4 h): Comprobación y ajuste de todos los elementos de retroalimentación visual y sonora para asegurar que las acciones del juego quedan siempre claras para el jugador.
- **Corrección de errores** (4 h): Detección y resolución de fallos o comportamientos inesperados identificados durante las pruebas finales del sistema.
- **Pruebas de integración** (2 h): Verificación del funcionamiento conjunto de todos los sistemas implementados a lo largo del proyecto, asegurando la coherencia y estabilidad del producto final.

Paralelamente al desarrollo técnico, se estimaron aproximadamente **140 horas** para la elaboración progresiva de la memoria del TFG, incluyendo la descripción del diseño del videojuego, la justificación de las decisiones técnicas adoptadas y el análisis de los resultados obtenidos.

En conjunto, la estimación total de dedicación para el proyecto asciende a unas **300 horas**, de las cuales 160 corresponden al desarrollo del videojuego distribuido entre las cinco iteraciones, y las 140 restantes a la documentación, realizada de forma paralela al desarrollo.

La Tabla 4.1 recoge de forma resumida las iteraciones planificadas, la estimación de horas de cada una y la fecha estimada de finalización dentro del cronograma del proyecto.

It.	Descripción	Horas est.	Fecha est. finalización
1	Implementación del juego base	20 h	2026-02-22
2	Interfaz de usuario y pulido visual	56 h	2026-04-05
3	Mejoras audiovisuales	15 h	2026-04-19
4	Implementación del bot inteligente	55 h	2026-05-24
5	Pulido final	14 h	2026-06-07
Documentación		140 h	2026-06-14
Total		300 h	

Tabla 4.1: Iteraciones planificadas, estimación de horas y fechas de finalización

La Figura 4.1 muestra de manera integral todas las iteraciones del proyecto junto con la tarea de documentación.

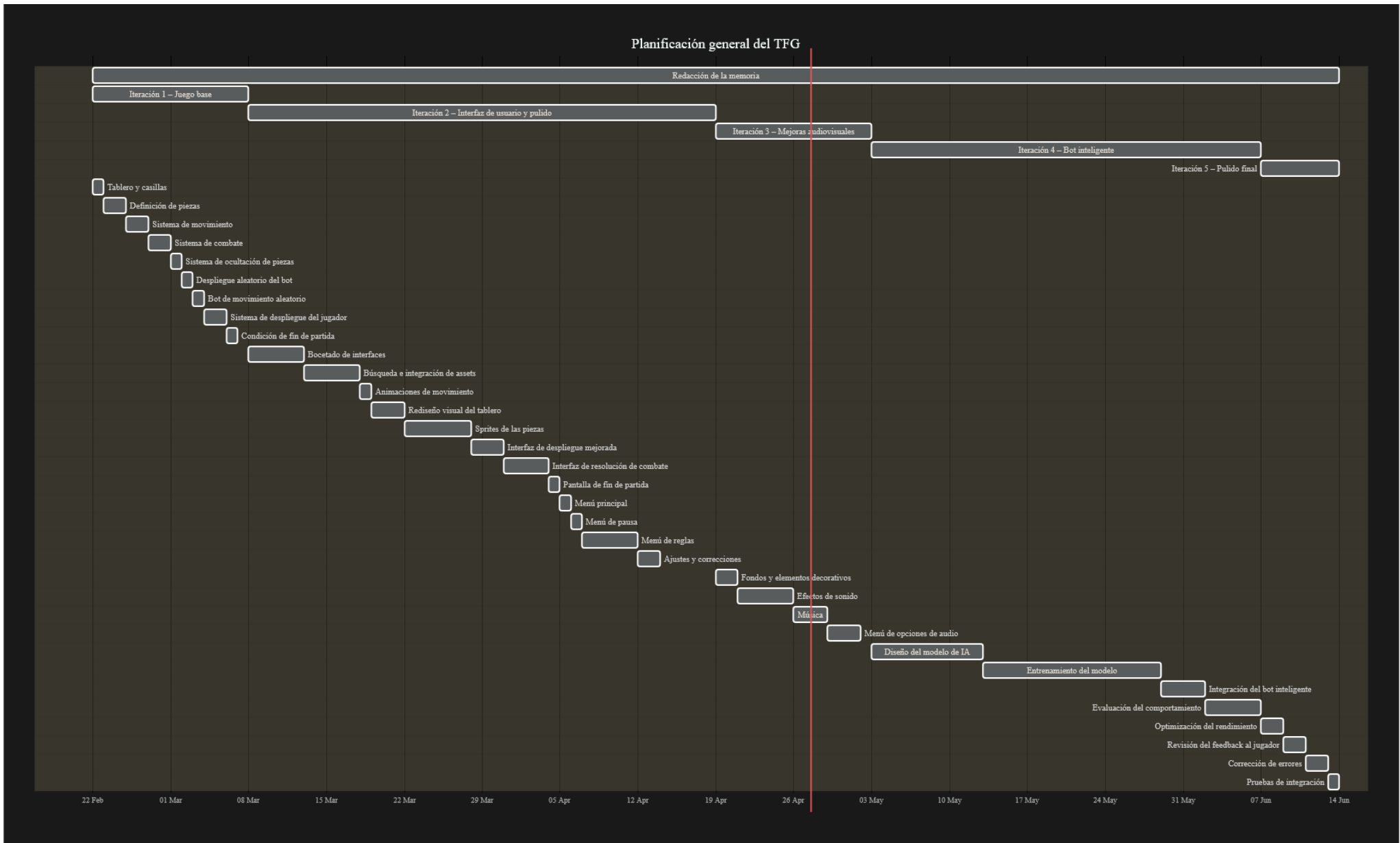


Figura 4.1: Diagrama de Gantt general del desarrollo del TFG, incluyendo todas las iteraciones y la documentación

4.2. Riesgos

Durante la fase de planificación se identificaron distintos riesgos que podrían afectar al correcto desarrollo del proyecto, recogidos en la Tabla 4.2. Para cada uno se ha analizado su naturaleza e impacto potencial, definiendo medidas de contingencia orientadas a minimizar sus efectos en caso de materializarse.

La identificación temprana de estos riesgos permite anticipar posibles problemas técnicos y organizativos, facilitando la toma de decisiones durante el desarrollo y contribuyendo a mantener la estabilidad del proyecto.

Riesgo	Tipo	Impacto	Plan de contingencia
R1: Fallos del motor o incompatibilidades	Técnico	Alto	Revisar dependencias y adaptar los scripts afectados.
R2: Retrasos en el cronograma	Temporal	Medio	Reajustar tareas no prioritarias y reducir funcionalidades accesorias.
R3: Errores en el bot o en las reglas del juego	Técnico	Medio	Simplificar el comportamiento de la IA y priorizar la estabilidad de las mecánicas fundamentales.
R4: Pérdida de datos o código fuente	Técnico	Alto	Uso de control de versiones y copias de seguridad en la nube.
R5: Falta de tiempo para pruebas y ajuste del juego	Temporal	Alto	Priorizar pruebas de jugabilidad y corregir únicamente los errores críticos.
R6: Carencia de recursos gráficos o sonoros adecuados	Logístico	Bajo	Crear recursos propios o adaptar recursos de terceros de libre uso.
R7: Incapacidad temporal para el desarrollo	Temporal	Alto	Reorganizar tareas, priorizar funcionalidades críticas y ajustar el cronograma.
R8: Dificultad en el diseño o implementación del bot inteligente	Técnico	Alto	Simplificar el modelo de decisión si fuera necesario; mantener operativo el bot aleatorio como versión mínima funcional.
R9: Bloqueos por depuración de errores difíciles	Técnico	Medio	Mantener control de versiones; realizar pruebas frecuentes; documentar los problemas encontrados y consultar la documentación oficial de Godot y C#.

Tabla 4.2: Riesgos identificados y planes de contingencia

4.3. Restricciones

El desarrollo de *NeuroForge* ha estado condicionado por una serie de restricciones que han influido directamente en la planificación, la selección de herramientas y el alcance final del proyecto.

- **Técnicas:** Se establece el uso del motor *Godot* junto con el lenguaje de programación *C#*. Aunque *Godot* emplea de forma nativa *GDScript*, se ha optado por *C#* debido a su soporte oficial y su mayor adecuación al perfil del proyecto. Asimismo, se limita el uso a librerías y recursos gratuitos o de código abierto.
- **Temporales:** El desarrollo se limita al periodo de realización del TFG, lo que impone una duración finita y condiciona directamente el número de funcionalidades que pueden implementarse.
- **Económicas:** No se dispone de presupuesto para la adquisición de licencias de software de pago, la contratación de colaboradores ni la compra de recursos gráficos o sonoros.
- **Personales:** El proyecto es desarrollado íntegramente por un único estudiante, lo que limita la paralelización de tareas y exige una gestión eficiente del tiempo disponible.
- **Académicas:** El trabajo debe ajustarse a la normativa, estructura y requisitos establecidos por la Escuela de Ingeniería Informática de Valladolid para los Trabajos de Fin de Grado.

4.4. Herramientas

Para el desarrollo del TFG se han empleado distintas herramientas de software tanto para la implementación del videojuego como para la gestión del proyecto y la redacción de la memoria. La selección de estas herramientas se ha realizado atendiendo a criterios de adecuación técnica, disponibilidad gratuita y facilidad de uso, buscando en todo momento optimizar el proceso de desarrollo. En la Tabla 4.3 se recoge un resumen de las principales herramientas utilizadas y su función dentro del proyecto.

El desarrollo del videojuego se ha realizado utilizando el motor *Godot*, descrito en detalle en el capítulo de antecedentes. En este proyecto se ha empleado como base para la implementación de la lógica del juego, la gestión del tablero, las interacciones del jugador y la representación visual del estado de la partida.

Para la programación de la lógica del juego se ha utilizado el lenguaje *C#*, que cuenta con soporte oficial dentro del motor *Godot*. La elección de este lenguaje se debe a su robustez, su tipado estático y su amplia adopción en el desarrollo de software, lo que facilita la organización del código y el mantenimiento del proyecto a medida que aumenta su complejidad.

Durante el desarrollo también se ha empleado un sistema de control de versiones basado en *Git*, utilizando la plataforma *GitHub* como repositorio remoto. Esta herramienta ha permitido

llevar un seguimiento de los cambios realizados en el código, mantener un historial de versiones del proyecto y disponer de una copia de seguridad del mismo en la nube.

En cuanto a la creación y edición de recursos visuales, se han utilizado herramientas de diseño gráfico como Aseprite y Paint. Estas aplicaciones han servido para generar o modificar sprites y otros elementos visuales necesarios para representar el tablero, las piezas y los distintos componentes de la interfaz del juego.

Para la gestión de recursos sonoros se han empleado herramientas como Audacity, junto con recursos procedentes de la plataforma Freesound. Audacity ha permitido realizar pequeñas ediciones o ajustes sobre efectos de sonido, mientras que Freesound se ha utilizado como fuente de recursos de audio de libre uso que pueden incorporarse al proyecto respetando sus licencias.

La redacción y maquetación de la memoria del TFG se ha realizado mediante $\text{\LaTeX}$, un sistema de preparación de documentos ampliamente utilizado en entornos académicos y científicos que facilita la estructuración del contenido, la generación automática de referencias y la gestión de figuras y tablas.

Finalmente, para la planificación y organización de las tareas del proyecto se ha utilizado Notion, una herramienta de gestión que ha permitido estructurar las distintas fases del desarrollo, definir hitos y realizar un seguimiento del progreso del trabajo a lo largo del tiempo. También se ha utilizado como borrador, tomar notas temporales sobre correcciones, llevar la cuenta de horas invertidas en el proyecto y como herramienta para cosas como el diagrama de Gantt.

Tipo	Herramienta	Uso principal
Motor de juego	Godot	Desarrollo del videojuego, lógica, interfaz...
Lenguaje de programación	C#	Programación de la lógica de juego y scripts.
Control de versiones	Git / GitHub	Gestión de versiones, control de cambios y copia de seguridad del código.
Diseño gráfico	Aseprite / Paint	Creación y edición de sprites, texturas y elementos visuales de la interfaz y el tablero.
Audio	Audacity / Freesound	Edición de efectos sonoros y obtención de música de fondo de libre uso.
Documentación	$\text{\LaTeX}$	Redacción y maquetación del presente documento del TFG.
Gestión de tareas	Notion	Planificación de fases, control de hitos y organización del flujo de trabajo.

Tabla 4.3: Herramientas

4.5. Variación de planificación

A lo largo del desarrollo del proyecto se han producido algunas variaciones respecto a la planificación inicial, aunque sin llegar a afectar de forma significativa al calendario ni al alcance del trabajo.

El cambio más relevante respecto a la planificación original fue la reorganización del orden de las iteraciones. En la planificación inicial se contemplaban cuatro iteraciones, en las que el desarrollo del bot inteligente se situaba en segundo lugar, antes del diseño de la interfaz de usuario. Sin embargo, durante el desarrollo se optó por invertir este orden, abordando primero la interfaz y posponiendo el bot a la cuarta iteración.

Esta decisión responde a una priorización deliberada del producto final, dado que el bot inteligente constituye un objetivo secundario del proyecto, se consideró más adecuado garantizar primero la existencia de un videojuego completo y funcional de principio a fin, con una experiencia de juego clara y pulida. De este modo, aunque el desarrollo del bot no llegara a completarse en el tiempo disponible, el proyecto contaría en todo caso con un resultado presentable y jugable, apoyado en el bot aleatorio ya implementado en la primera iteración como oponente mínimo funcional.

Como consecuencia de esta reorganización, la planificación pasó de cuatro a cinco iteraciones, separando las mejoras audiovisuales en una iteración propia. Este ajuste no supuso un incremento en el tiempo total estimado, sino una redistribución del trabajo en fases más cohesionadas.

En cuanto a las desviaciones temporales, las iteraciones completadas hasta la fecha se han desarrollado dentro de los márgenes previstos, sin retrasos significativos respecto al cronograma establecido. La duración real de cada fase ha sido comparable a la estimación inicial, con variaciones menores atribuibles a la diferente dificultad de tareas concretas dentro de cada iteración.

(Escribir mas variaciones cuando se den)

...

4.6. Costes del proyecto

El desarrollo de este TFG se ha realizado en un contexto académico, por lo que no se han producido costes económicos directos asociados al proyecto. Todas las herramientas empleadas han sido gratuitas, de código abierto o disponibles mediante licencias académicas proporcionadas por la universidad, y el espacio de trabajo utilizado ha sido el domicilio particular propio.

No obstante, desde un punto de vista hipotético, si este proyecto se hubiera desarrollado en un entorno profesional, se habrían incurrido en los siguientes costes, distribuidos en tres categorías: costes de personal, software e infraestructura.

Costes de personal

El principal coste del proyecto corresponde al tiempo de trabajo invertido en su desarrollo. Según datos de Glassdoor (2025), el salario medio de un desarrollador junior de videojuegos en España es de aproximadamente **30.815 €** anuales, lo que equivale a un coste horario de unos **15 €/h** considerando una jornada laboral estándar de 160 horas mensuales [22].

La dedicación total estimada para el proyecto ha sido de **300 horas**, distribuidas entre las fases de diseño, implementación, pruebas y documentación. El coste de personal se calcula como:

$$C_{\text{personal}} = 300 \text{ h} \times 15,00 \text{ €/h} = 4,500,00 \text{ €}$$

Costes de *software*

Las herramientas de *software* utilizadas en el proyecto han sido en su mayoría gratuitas o disponibles mediante licencias académicas. La única herramienta de pago considerada es *Aseprite*, empleada para la creación y edición de sprites en *pixel art*, cuyo precio de adquisición es de **16,79 €**. No obstante, en el contexto real del proyecto se ha utilizado una licencia previamente adquirida, por lo que no ha supuesto un coste adicional.

El resto de herramientas (*Godot*, *Git/GitHub*, *Audacity*, $\text{\LaTeX}$, *Notion* y la licencia de *Astah Professional*) son de uso gratuito o han sido cedidas mediante licencia de estudiante por la universidad, por lo que su coste añadido al proyecto es nulo.

$$C_{\text{software}} = 16,79 \text{ €}$$

Costes de infraestructura

Los costes de infraestructura comprenden la amortización del equipo informático y los gastos derivados del uso del espacio de trabajo doméstico durante el periodo de desarrollo, estimado en **4 meses**.

Amortización del hardware: El equipo utilizado incluye el ordenador y sus periféricos (monitores y demás componentes), con un valor de reposición actual estimado en **2.000 €** y una antigüedad de **8 años**. Considerando una mínima vida útil total estimada de **10 años**, la cuota de amortización anual es:

$$A_{\text{anual}} = \frac{2,000 \text{ €}}{10 \text{ años}} = 200,00 \text{ €/año}$$

El coste añadido al proyecto, correspondiente a 4 meses de uso, es:

$$C_{\text{hardware}} = 200,00 \text{ €/año} \times \frac{4}{12} = 66,67 \text{ €}$$

Electricidad: El coste es de aproximadamente unos 70 €/mes y estimando que el uso del equipo informático representa aproximadamente un 20 % del consumo total del hogar, el coste eléctrico añadido al proyecto es:

$$C_{\text{luz}} = 70,00 \text{ €/mes} \times 20 \% \times 4 \text{ meses} = 56,00 \text{ €}$$

Conexión a internet: El coste ronda en torno a los 20 € al mes. El coste añadido al periodo de desarrollo es:

$$C_{\text{internet}} = 20,00 \text{ €/mes} \times 4 \text{ meses} = 80,00 \text{ €}$$

El coste total de infraestructura es:

$$C_{\text{infraestructura}} = 66,67 + 56,00 + 80,00 = 202,67 \text{ €}$$

Resumen de costes

La Tabla 4.4 recoge el desglose completo de los costes estimados del proyecto en un contexto profesional.

Categoría	Concepto	Coste estimado
Personal	Desarrollo (300 h $\times$ 15,00 €/h)	4.500,00 €
Software	Licencia Aseprite	16,79 €
Infraestructura	Amortización hardware (4 meses)	66,67 €
	Electricidad (4 meses)	56,00 €
	Conexión a internet (4 meses)	80,00 €
Total estimado		4.719,46 €

Tabla 4.4: Desglose de costes estimados del proyecto en contexto profesional

El coste real del proyecto en el contexto académico en el que se ha desarrollado ha sido **0 €**, al haberse utilizado exclusivamente herramientas gratuitas, licencias de estudiante y recursos propios ya disponibles.

Análisis

5.1. Actores y roles

En este apartado se identifican los actores que interactúan con el sistema, entendiendo como actor toda entidad externa que inicia o participa en la ejecución de casos de uso.

El único actor del sistema es el **Jugador**, que corresponde a la persona que interactúa con el videojuego a través de la interfaz gráfica mediante el ratón. Es el actor principal y exclusivo, responsable de iniciar y configurar la partida, desplegar sus piezas en la fase inicial, realizar movimientos y ataques durante su turno, y pausar o abandonar la partida en cualquier momento. El resto de la lógica del juego, incluyendo las acciones del oponente, es gestionada internamente por el propio sistema sin intervención externa.

5.2. Modelado de Requisitos

Los requisitos describen las condiciones, capacidades y restricciones que debe cumplir un sistema software para satisfacer las necesidades de sus usuarios y los objetivos de la organización. Se distinguen principalmente tres tipos de requisitos: los requisitos funcionales (RF), que especifican los servicios y comportamientos que debe ofrecer el sistema; los requisitos no funcionales (RNF), que establecen restricciones y propiedades de calidad como rendimiento, fiabilidad o usabilidad; y los requisitos de información (RI), que definen los datos que el sistema debe almacenar y gestionar para proporcionar dichos servicios.

Requisitos funcionales

- **RF01:** El sistema permitirá iniciar una nueva partida desde un estado inicial controlado, inicializando el tablero, las piezas y el turno inicial conforme a las reglas del juego.
- **RF02:** El sistema permitirá al jugador colocar sus piezas al inicio de la partida dentro de la zona de despliegue definida.
- **RF03:** El sistema gestionará los turnos alternos entre el jugador y el bot, impidiendo la realización de acciones fuera de turno.
- **RF04:** El sistema validará las acciones realizadas por el jugador, evitando movimientos o ataques no permitidos por las reglas del juego.
- **RF05:** El sistema revelará la identidad de las piezas implicadas cuando se produzca un combate, manteniendo visible la información revelada durante el resto de la partida.
- **RF06:** El bot tomará decisiones de movimiento y ataque de forma autónoma durante su turno, respetando las reglas y restricciones del juego.
- **RF07:** El sistema mantendrá y actualizará el estado de la partida durante su desarrollo, gestionando turnos, acciones válidas y transiciones entre estados.
- **RF08:** El sistema determinará automáticamente el final de la partida cuando se cumpla una condición de victoria o derrota, mostrando el resultado al jugador.
- **RF09:** La interfaz mostrará información visual y sonora relacionada con las acciones realizadas, los combates y los cambios de estado de la partida.
- **RF10:** La interfaz del juego será completamente manejable mediante ratón.
- **RF11:** El sistema permitirá al jugador pausar o abandonar la partida en curso de forma controlada.
- **RF12:** El sistema brindará al jugador con la información necesaria para entender las reglas del juego en caso de no conocerlas.

Requisitos no funcionales

- **RNF01:** El sistema deberá ser compatible con los sistemas operativos Windows 10 y Windows 11.
- **RNF02:** El videojuego deberá ejecutarse correctamente en un equipo con las siguientes especificaciones mínimas de hardware:
 - Procesador: CPU de doble núcleo a 2,0 GHz o superior.
 - Memoria RAM: 8 GB.
 - Tarjeta gráfica: GPU compatible con OpenGL 3.3 o superior.

- Almacenamiento: 200 MB de espacio libre en disco.
- **RNF03:** El sistema deberá responder a las acciones del jugador en un tiempo máximo de 2 segundos en condiciones normales de ejecución, garantizando una experiencia de juego fluida.
- **RNF04:** La interfaz deberá ser suficientemente clara e intuitiva para que un jugador sin experiencia previa en el juego pueda comprender las acciones disponibles en cada momento sin necesidad de consultar las reglas.

Requisitos de información

- **RI01 — Partida:** El sistema gestionará una sesión de juego almacenando la siguiente información:
 - Referencia al tablero de juego.
 - Turno actual (Jugador o Bot).
 - Estado de la partida (Fase de despliegue, turno del jugador, fin de la partida...).
 - Número de turno.
- **RI02 — Tablero:** El sistema almacenará la información relativa al tablero de juego:
 - Conjunto de casillas que conforman el tablero.
- **RI03 — Casilla:** Cada casilla del tablero almacenará la siguiente información:
 - Posición en la cuadrícula (x, y) .
 - Tipo (Pasable, no pasable...).
 - Pieza que la ocupa, si la hubiera.
- **RI04 — Pieza:** El sistema almacenará la siguiente información para cada pieza presente en la partida:
 - Tipo de pieza (Energy core, turret, scout...).
 - Rango numérico que determina el resultado de los combates.
 - Propietario (Jugador o Bot).
 - Si la pieza puede moverse o no.
 - Si es visible para el jugador.
 - Si el bot conoce la identidad de la pieza.
 - Casilla del tablero en la que se situa.
 - Historial de los últimos tres movimientos realizados como origen-destino.

5.3. Casos de uso

En este apartado se describen los casos de uso del sistema, que representan las interacciones posibles entre el actor y el sistema para alcanzar un objetivo concreto. Cada caso de uso recoge el flujo normal de acciones, las posibles excepciones y los requisitos funcionales asociados. La Figura 5.1 muestra el diagrama general de casos de uso, donde se reflejan las relaciones entre el actor, los casos de uso principales y las dependencias entre ellos.

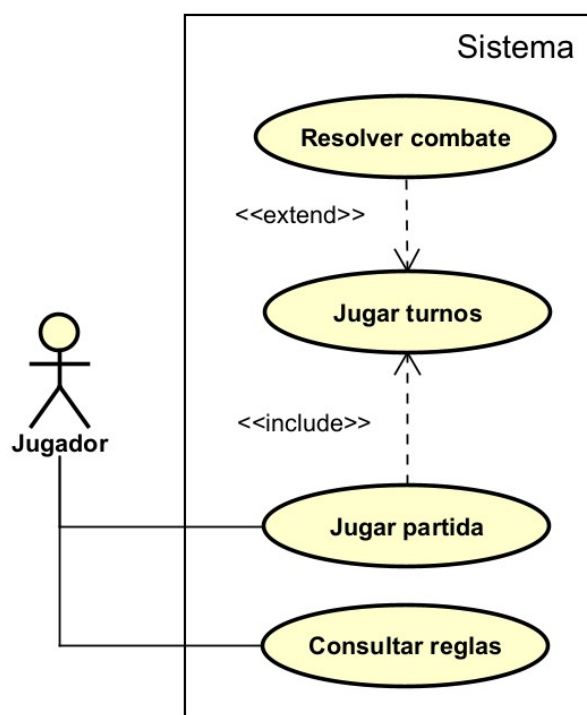


Figura 5.1: Diagrama de casos de uso

5.3.1. Lista de casos de uso

ID	Nombre	Descripción
CU-01	Jugar partida	El sistema permite al jugador llevar a cabo una partida completa contra el bot, desde la fase de despliegue hasta la resolución final.
CU-02	Resolver combate	El Sistema gestiona un ronda completa, la acción del jugador, la respuesta del bot y la comprobación de condiciones de victoria hasta que la partida acabe.
CU-03	Resolver combate	El Sistema gestiona el combate entre piezas de manera automática con su respectiva interfaz.
CU-04	Consultar reglas	El sistema permite al jugador consultar las reglas dentro del juego.

Tabla 5.1: Lista de casos de uso del sistema

CU-01: Jugar partida

ID	CU-01
Nombre	Jugar partida
Actor	Jugador
Descripción	El Sistema permite al jugador llevar a cabo una partida completa contra el bot, desde la fase de despliegue hasta la resolución final.
Req. relacionados	RF01, RF02., RF09, RF10, RF11 .
Pre-condición	El sistema se encuentra mostrando la escena del menú principal
Secuencia normal	1. El Jugador selecciona la opción de jugar una partida. 2. El Sistema cambia la escena a la de la partida, inicia el tablero y muestra la interfaz de despliegue. 3. El Jugador coloca todas sus piezas en el tablero. 4. El Jugador selecciona comenzar la partida. 5. El Sistema coloca todas las piezas del bot e inicia la partida. 6. Se inicia CU-02 (Jugar turnos). 7. El Sistema muestra la interfaz del final de partida. 8. El Jugador selecciona volver al menú principal. 9. El Sistema cambia la escena a la del menú principal.
Flujos alternativos	4.a. No estan todas las piezas colocadas → Regresa al paso 3. 8.a. El Jugador selecciona jugar otra partida → Regresa al paso 2.
Post-condición	La partida termina y el jugador vuelve al menú principal.

Tabla 5.2: CU-01: Jugar partida

CU-02: Jugar turnos

ID	CU-02
Nombre	Jugar turnos
Actor	Jugador
Descripción	El Sistema gestiona un ronda completa, la acción del jugador, la respuesta del bot y la comprobación de condiciones de victoria hasta que la partida acabe.
Req. relacionados	RF03, RF04, RF06, RF07, RF08, RF10.
Pre-condición	El Sistema se encuentra en la escena de la partida y esta esperando input del Jugador.
Secuencia normal	1. El Jugador selecciona una de sus piezas con posibilidad de movimiento. 2. El Sistema resalta las casillas válidas a las que se puede mover la pieza seleccionada. 3. El Jugador selecciona una casilla resaltada. 4. El Sistema mueve la pieza a la casilla seleccionada. 5. El Sistema comprueba que la partida no se ha terminado. 6. El Sistema ejecuta el turno del bot, moviendo una de sus piezas válidas. 7. El Sistema comprueba que la partida se ha terminado.
Flujos alternativos	1.a. El Jugador no puede mover ninguna pieza → el Sistema da la partida por terminada y el caso de uso finaliza. 1.b. El Jugador no selecciona una pieza válida → el Sistema ignora la acción y regresa al paso 1. 3.a. El Jugador no selecciona una casilla válida a la que puede moverse la pieza → El Sistema ignora la acción, quita el resaltado de las casillas que lo estén y regresa al paso 1. 4.a. La pieza ha hecho un movimiento característico de solo esa pieza → El Sistema revela la identidad de esa pieza y continua. 4.b. La casilla destino esta ocupada por una pieza del bot → Se inicia CU-03 (Resolver combate). 5.a. La partida ha terminado → El caso de uso finaliza. 6.a. La pieza ha hecho un movimiento característico de solo esa pieza → El Sistema revela la identidad de esa pieza y continua. 6.b. La casilla destino esta ocupada por una pieza del jugador → Se inicia CU-03 (Resolver combate). 7.a. La partida no ha terminado → Regresa al paso 1.
Post-condición	La partida finaliza y ya no se realizan mas turnos.

Tabla 5.3: CU-01: Jugar partida

CU-03: Resolver combate

ID	CU-03
Nombre	Resolver combate
Actor	Jugador
Descripción	El Sistema gestiona el combate entre piezas de manera automática con su respectiva interfaz.
Req. relacionados	RF05, RF09.
Pre-condición	Durante el transcurso de un turno se ha movido una pieza a una casilla en la que se situa otra del bando opuesto.
Secuencia normal	1. El Sistema muestra la interfaz de combate. 2. El Sistema reproduce el combate y da el resultado. 3. El Sistema revela las piezas ocultas 4. El Jugador una vez visto el resultado, solicita continuar con la partida. 5. El Sistema cierra la interfaz de combate. 6. El Sistema elimina la pieza (o piezas) perdedora en el combate del tablero.
Flujos alternativos	3.a. No hay piezas ocultas → El Sistema ignora este paso y continua al paso 4
Post-condición	El Sistema continua con la partida en el lugar previo al combate.

Tabla 5.4: CU-03: Resolver combate

CU-04: Consultar reglas

ID	CU-04
Nombre	Consultar reglas
Actor	Jugador
Descripción	El sistema permite al jugador consultar las reglas dentro del juego.
Req. relacionados	RF12.
Pre-condición	El Sistema esta en la escena del menu de pausa o en el menu principal.
Secuencia normal	1. El Jugador solicita consultar las reglas 2. El Sistema muestra el menu con las reglas 3. El Jugador consulta las reglas 4. El Jugador solicita salir del menu de reglas 5. El Sistema cierra el menu de reglas.
Flujos alternativos	■ N/A
Post-condición	El Sistema continua con con normalidad en la escena en la que se consulto las reglas.

Tabla 5.5: CU-04: Consultar reglas

5.4. Modelo de dominio

El modelo de dominio es una representación visual de las entidades conceptuales (objetos del mundo real o del sistema) y las relaciones que existen entre ellas dentro del contexto del videojuego. Su objetivo es identificar los conceptos clave, sus atributos y cómo interactúan, sirviendo como puente entre los requisitos funcionales y el diseño técnico final.

A continuación, en la Figura 5.2, se presenta el diagrama de clases de dominio que estructura la lógica del juego.

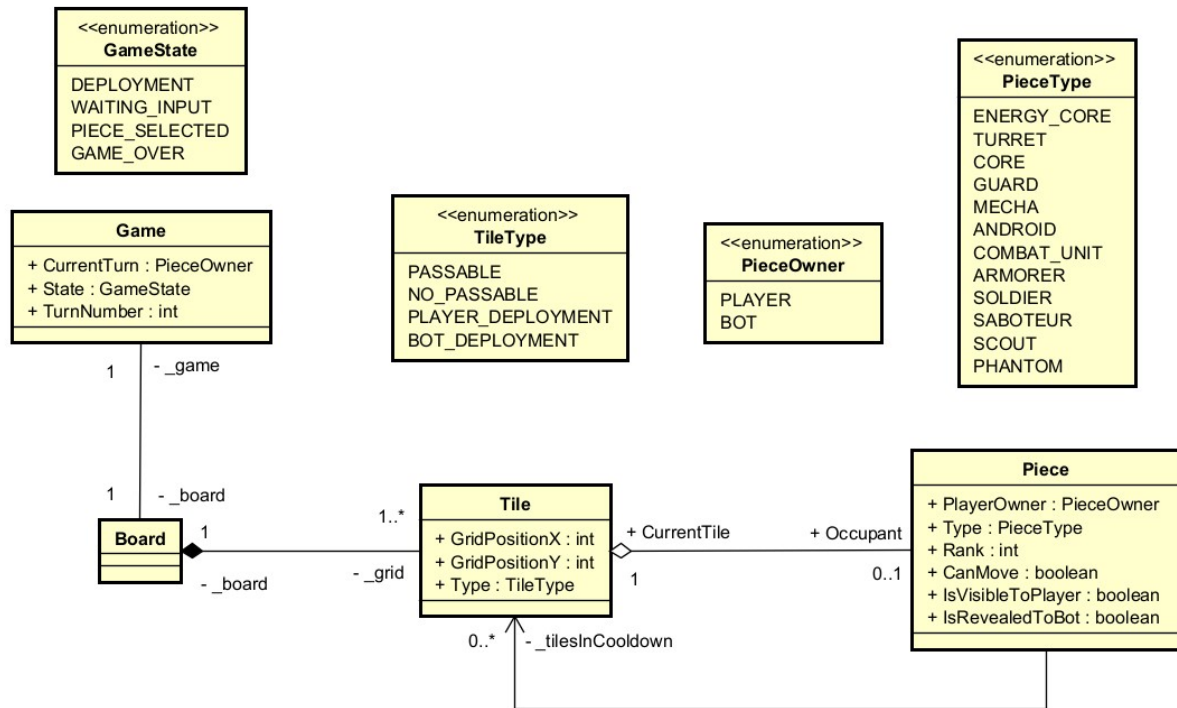


Figura 5.2: Diagrama del modelo de dominio

5.5. Modelo de proceso de negocio

El modelo de proceso de negocio describe el flujo detallado de actividades y decisiones que tienen lugar durante la ejecución del sistema. Mientras que los casos de uso definen *qué* hace el sistema, los diagramas de actividad reflejan *cómo* se ejecutan dichas interacciones, mostrando la secuencia lógica, las bifurcaciones y el procesamiento de datos.

A continuación, se detallan los flujos de actividad correspondientes a los casos de uso principales del sistema, proporcionando una visión dinámica de la lógica del videojuego.

5.5.1. Actividad: Jugar partida (CU-01)

Este flujo describe el ciclo de vida completo de una sesión de juego, desde la interacción inicial en el menú hasta la finalización de la experiencia. En la Figura 5.3 se detalla la secuencia que incluye la preparación del escenario, la fase crítica de despliegue de piezas y la transición hacia el núcleo jugable de turnos.

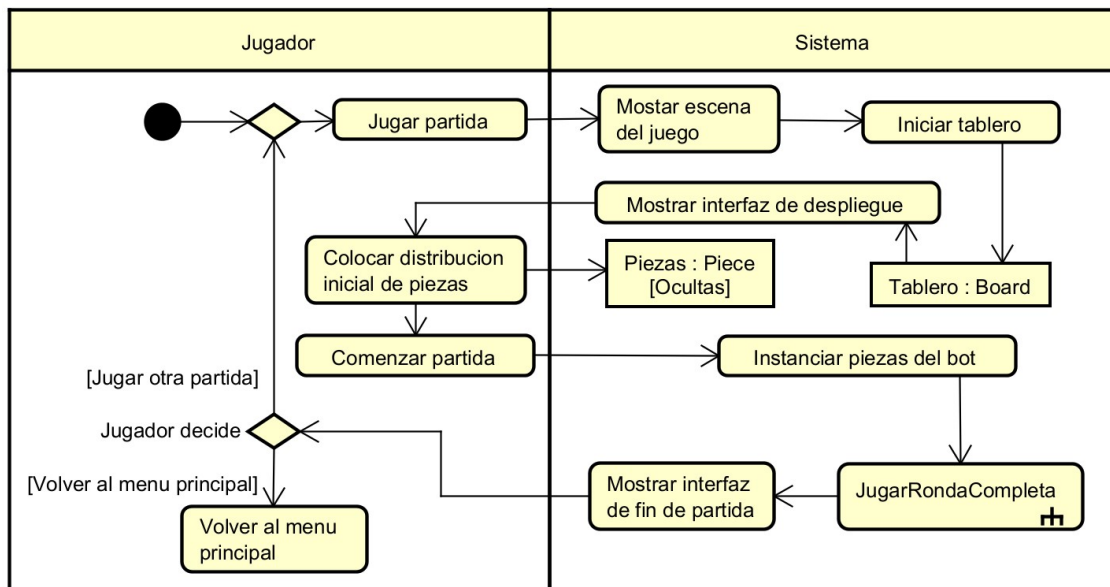


Figura 5.3: Diagrama de actividad: Jugar partida (CU-01)

5.5.2. Actividad: Jugar turnos (CU-02)

La lógica central del juego se basa en un ciclo de estados donde el control alterna entre el usuario y la inteligencia artificial. Como se observa en la Figura 5.4, este proceso gestiona la selección de unidades, la validación de movimientos en el tablero y las comprobaciones automáticas de fin de partida tras cada acción.

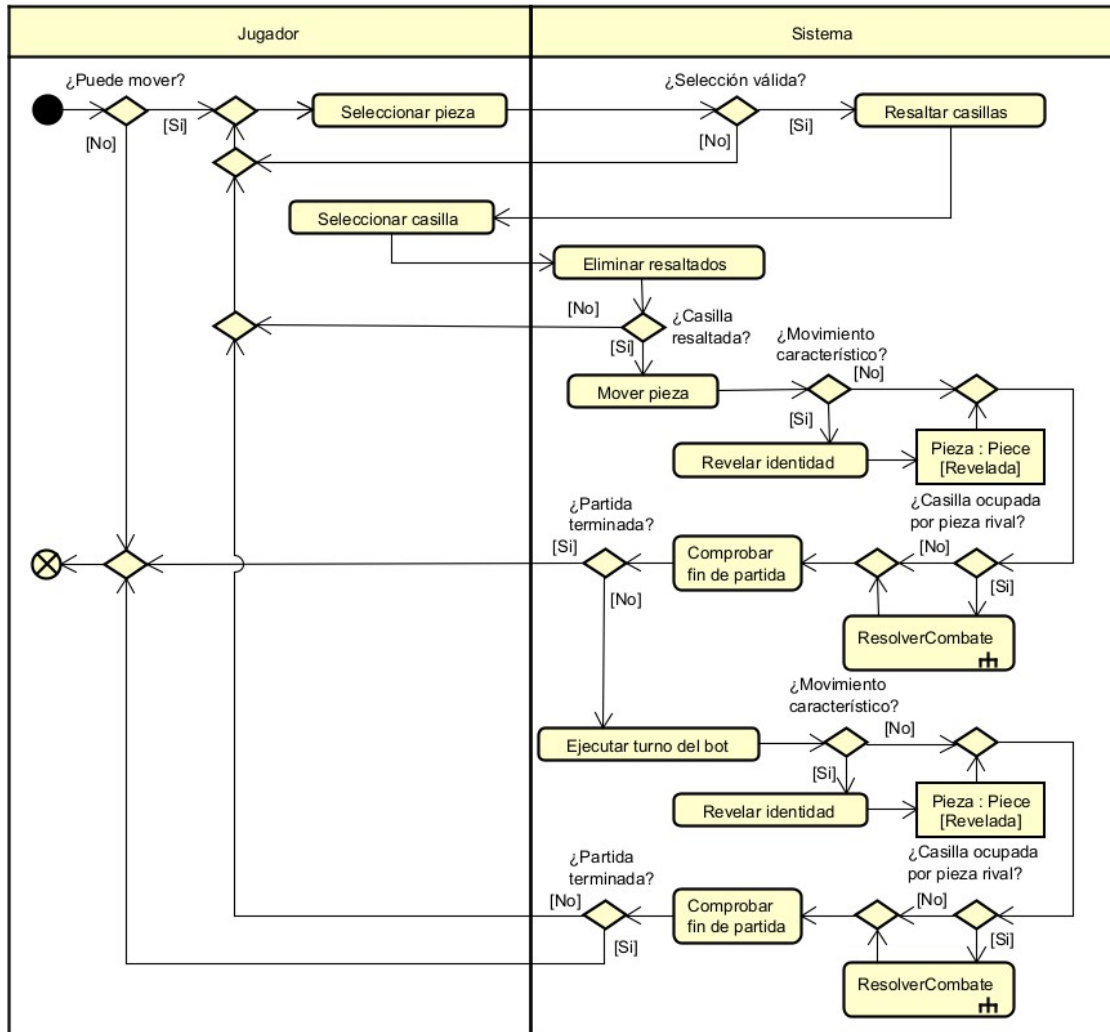


Figura 5.4: Diagrama de actividad: Jugar turnos (CU-02)

5.5.3. Actividad: Resolver combate (CU-03)

El combate es una subrutina automática que se activa por la coincidencia de posiciones en el tablero. El diagrama representado en la Figura 5.5 muestra cómo el sistema procesa la comparación de rangos de las piezas, actualiza la visibilidad de las mismas para el jugador y modifica el estado del tablero eliminando las bajas resultantes.

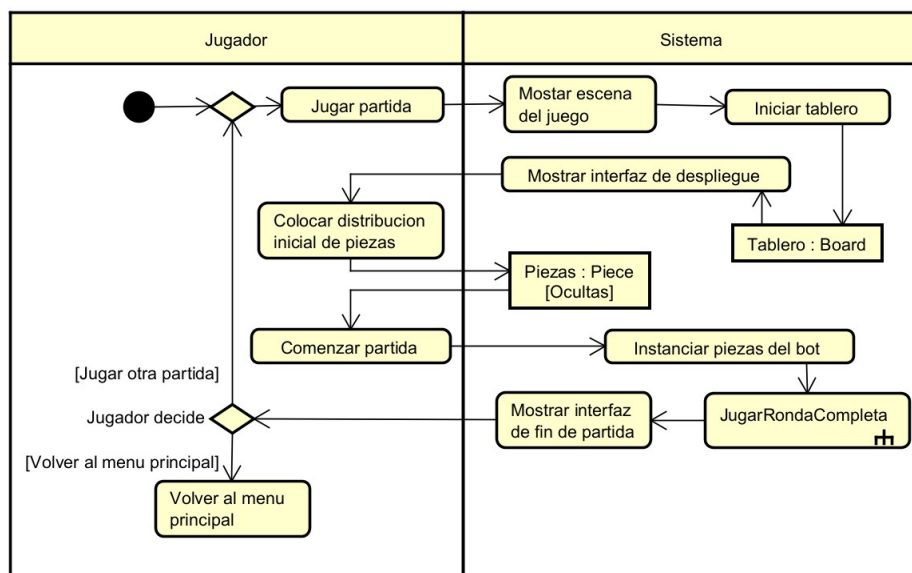


Figura 5.5: Diagrama de actividad: Resolver combate (CU-03)

5.5.4. Actividad: Consultar reglas (CU-04)

Este flujo representa una interacción de apoyo que no afecta al estado de la partida pero garantiza la usabilidad. La Figura 5.6 ilustra la capacidad del sistema para pausar visualmente el contexto actual y presentar la documentación necesaria al jugador antes de retornar al estado previo.

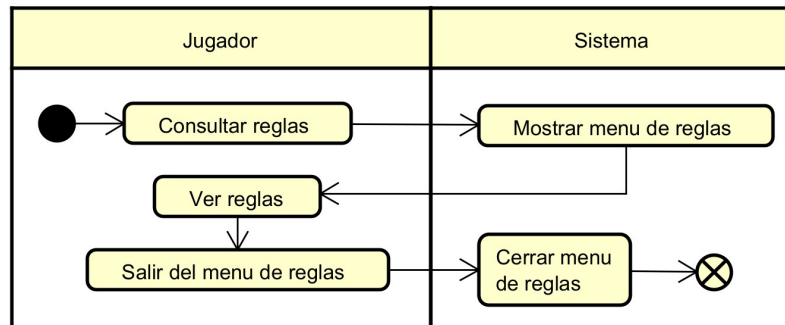


Figura 5.6: Diagrama de actividad: Consultar reglas

Diseño

En este capítulo se describe el diseño técnico de NeuroForge a partir del código implementado. Se aborda la arquitectura general del sistema, los módulos que lo componen, el diseño de clases, la interfaz de usuario, los patrones de diseño aplicados y las consideraciones de seguridad y rendimiento.

6.1. Arquitectura del sistema

El modelo de ejecución de Godot

Para comprender las decisiones de diseño adoptadas en este proyecto es necesario entender primero cómo organiza Godot el contenido de un juego. Godot estructura toda la lógica y los elementos visuales en un **árbol de escenas** (*SceneTree*): una jerarquía de objetos llamados **nodos**, donde cada nodo representa una unidad funcional del juego, ya sea un personaje, una casilla del tablero, un menú o el gestor de partida.

Cada nodo hereda, en última instancia, de la clase base **Node**. Esta es la unidad mínima funcional del motor; no posee posición ni representación visual, pero define el ciclo de vida compartido por todos los objetos del árbol: el método **Ready()** se ejecuta cuando el nodo entra al árbol y es el punto habitual de inicialización, mientras que **Process()** se invoca en cada fotograma para la lógica continua.

A partir de esta base, el proyecto utiliza especializaciones más complejas que expanden las capacidades del nodo raíz. Este mecanismo de **herencia** permite que, por ejemplo, **Node2D** añada propiedades de posición, rotación y escala para elementos en el mundo 2D (como las piezas y el tablero), mientras que **Control** se especializa en interfaces de usuario con gestión de anclajes y márgenes. Gracias a esta estructura, incluso los componentes de lógica pura pueden integrarse en el ciclo de vida del motor heredando simplemente de **Node**, evitando así la sobrecarga de datos espaciales que no requieren.

Dicha jerarquía de nodos se organiza de forma práctica mediante **escenas**, que funciona como un plano reutilizable compuesto por una rama de nodos. Una escena permite agrupar estos nodos especializados para que trabajen como una única entidad lógica dentro del árbol principal del juego.

Un ejemplo representativo en NeuroForge es la escena **Tile** (ver Figura 6.1), encargada de las casillas del tablero. Como se observa en la imagen, el nodo raíz es un **Area2D**; este nodo no solo hereda las capacidades espaciales de **Node2D**, sino que aporta funcionalidades propias, como por ejemplo, permitir detectar eventos de ratón y colisiones. Bajo él, la escena se compone de hijos específicos: un **CollisionShape2D** para delimitar el área de interacción de dicho área (Específicamente los **Area2D** necesitan un **CollisionShape2D** hijo para funcionar) y nodos **Sprite2D** para la representación visual. Esta composición ilustra cómo la suma de nodos permite construir objetos más complejos y funcionales.

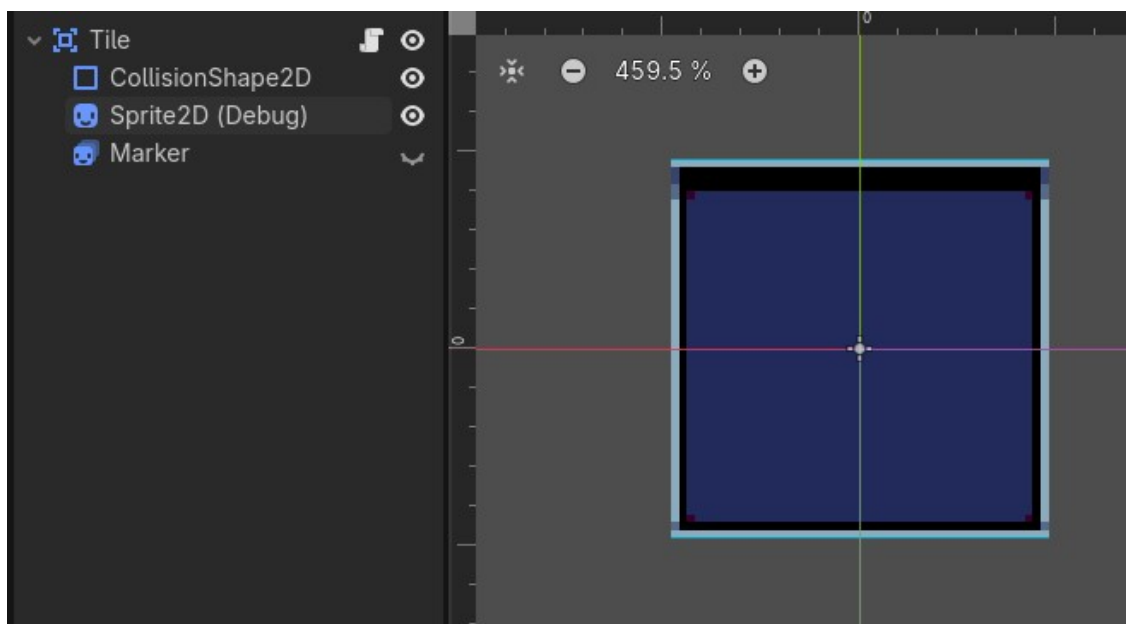


Figura 6.1: Estructura de la escena **Tile** en el editor de Godot

La comunicación entre nodos puede realizarse de tres formas. La primera es mediante **referencias directas**: un nodo obtiene acceso a otro con `GetNode()` y llama a sus métodos directamente. La segunda es mediante **señales**, el sistema de eventos nativo de Godot: un nodo emite una señal cuando ocurre algo relevante y otros nodos suscritos reaccionan sin necesidad de conocerse mutuamente. Para esta última he optado por utilizar los eventos de C# estándar (event Action), utilizados para una comunicación más rápida y con tipado fuerte entre clases de lógica pura y controladores, reduciendo así la dependencia con el editor de Godot. La tercera es mediante **clases estáticas o instancias globales**, accesibles desde cualquier punto del código sin depender de la posición en el árbol. Una característica importante de Godot con C# es que no toda clase tiene que ser un nodo: es válido definir clases C# ordinarias que operen sobre los datos del árbol sin pertenecer a él, lo que permite aislar la lógica de negocio del ciclo de vida del motor.

La Figura 6.2 ilustra este modelo con un subconjunto representativo de los nodos del proyecto. No pretende recoger la totalidad del árbol de escenas, sino mostrar algunos tipos de nodos más

relevantes, su jerarquía y algunos de los tres mecanismos de comunicación descritos.

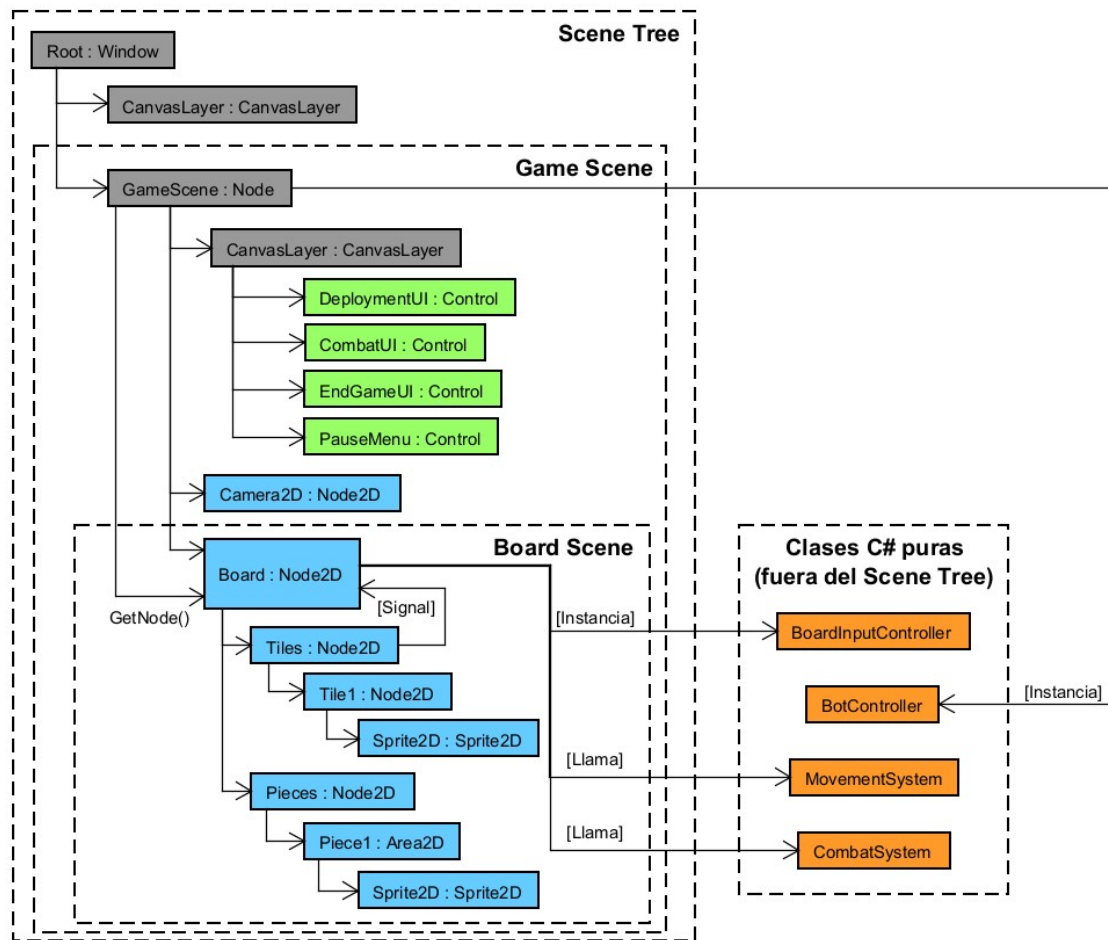


Figura 6.2: Modelo de ejecución de Godot ilustrado con algunos nodos del proyecto.

Arquitectura de NeuroForge

Con este modelo en mente, el sistema se organiza en una **arquitectura en tres capas** que separa la representación visual de la lógica pura de juego y de los datos. Esta separación es una decisión de diseño para evitar que las reglas del juego queden entrelazadas con el código de renderizado o de interfaz.

La Figura 6.3 muestra una vista simplificada de los principales componentes de cada capa. Se incluyen los elementos más representativos de cada nivel.

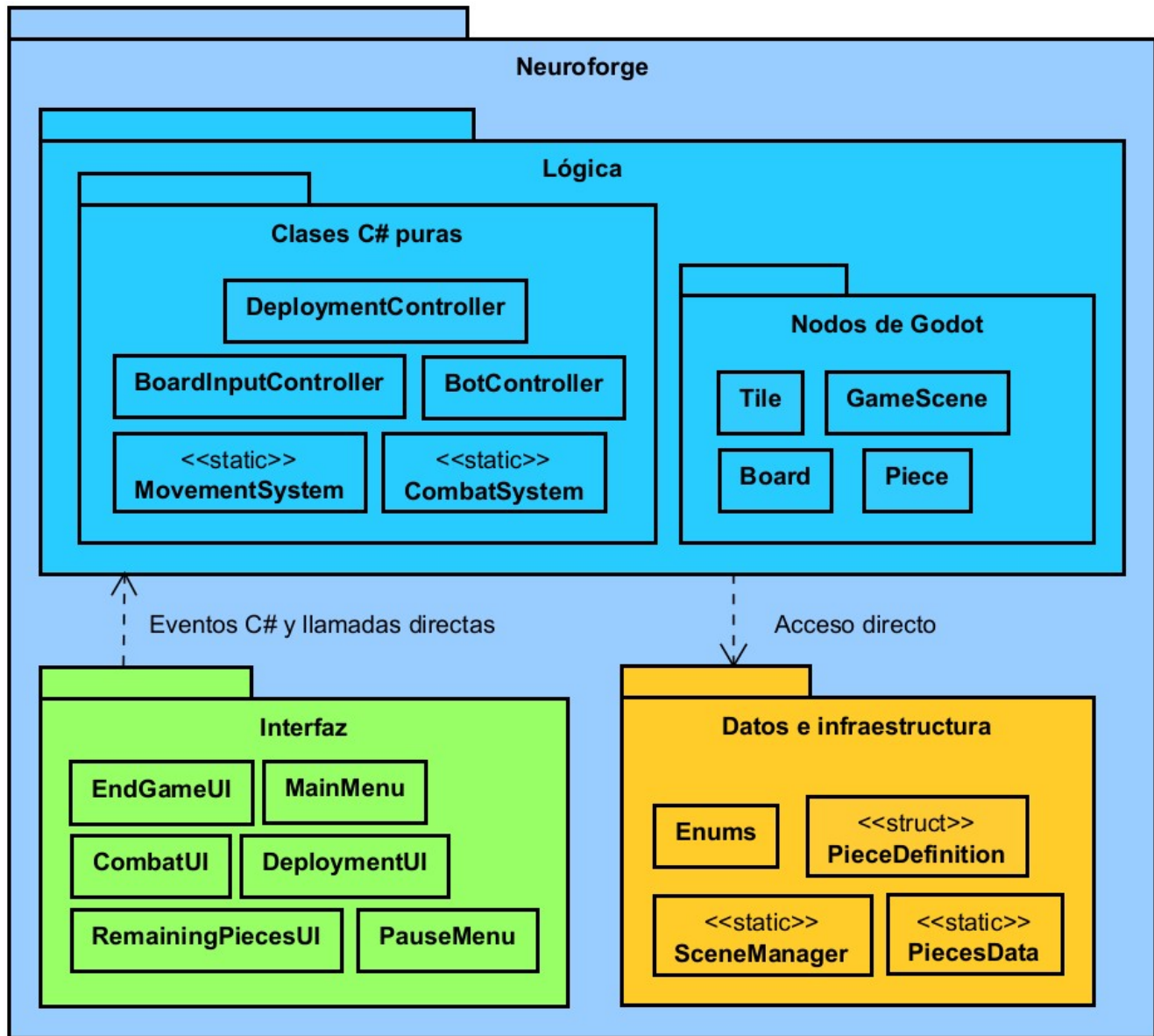


Figura 6.3: Vista simplificada de la arquitectura de NeuroForge en tres capas.

La **capa de interfaz** agrupa todos los nodos que heredan de `Control`. Su responsabilidad se limita a capturar las intenciones del jugador y reflejar el estado del juego visualmente. Ninguno de estos nodos toma decisiones de lógica de juego: cuando el jugador pulsa un botón, la capa de presentación notifica a la capa inferior mediante eventos C# estándar (`event Action`) o señales de Godot (`[Signal]`), y espera recibir los datos actualizados para refrescar la vista.

La **capa de lógica de juego** es el núcleo funcional del sistema. Aquí conviven dos tipos de clases con naturalezas distintas. `GameScene`, `Board`, `Tile` y `Piece` son nodos de Godot porque necesitan estar en el árbol de escenas para gestionar el input del ratón, coordinar animaciones y participar en el ciclo de vida del motor. Por otro lado, `BoardInputController`, `DeploymentController` y `BotController` son clases C# puras creadas e instanciadas desde `GameScene` o `Board`, independientes del motor y por tanto más ligeras y fáciles de reutilizar. `MovementSystem` y `CombatSystem` son clases estáticas puras sin estado.

La **capa de datos e infraestructura** comprende los datos estáticos de configuración (`PiecesData`), el gestor de navegación entre escenas (`SceneManager`) y los enums que definen

los valores del dominio (`GameState`, `PieceType`, `TileType`, etc.).

Comunicación entre capas

Las tres capas se comunican siguiendo un flujo definido que evita dependencias circulares. La Figura ?? muestra el flujo concreto que ocurre cuando el jugador confirma el despliegue de sus piezas e inicia la partida, ilustrando los dos mecanismos principales de comunicación: eventos `C#` de presentación hacia lógica, y llamadas directas en el sentido contrario.

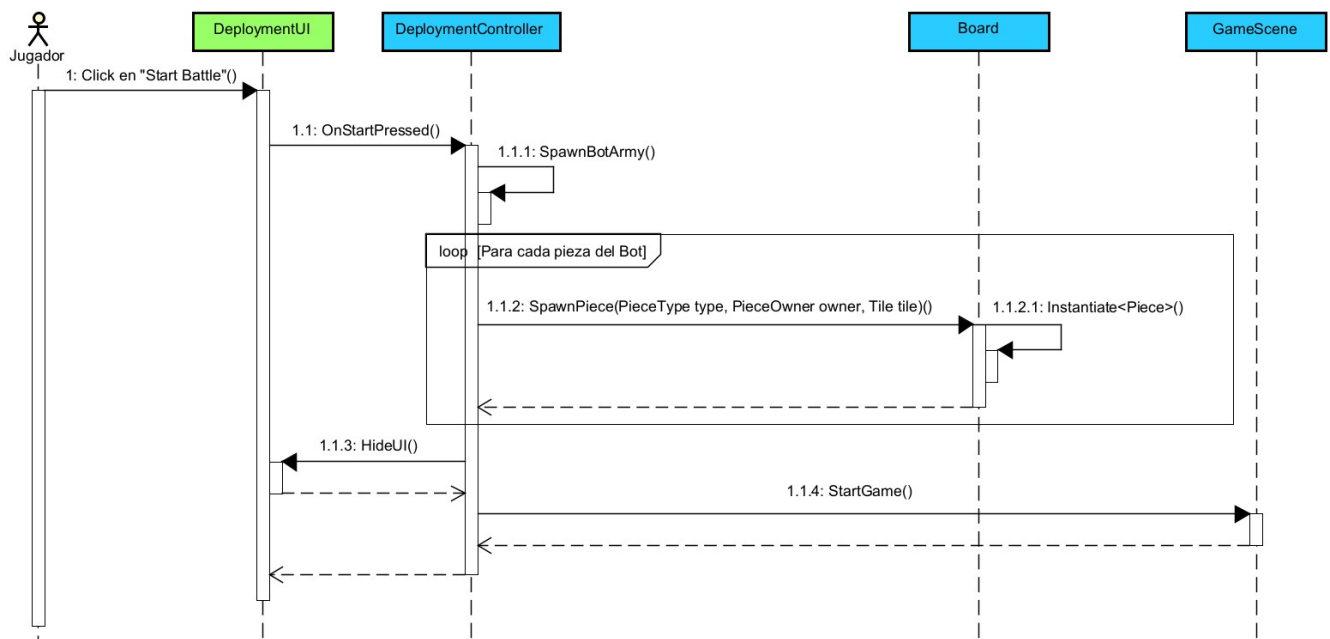


Figura 6.4: Ejemplo de comunicación entre capas durante la confirmación del despliegue.

La presentación se comunica hacia la lógica mediante **eventos `C#`**: `DeploymentUI` emite `OnStartPressed` sin conocer quién lo escucha. En sentido contrario, la lógica actualiza la presentación mediante **llamadas directas** a métodos de la UI como `HideUI()`. Dentro de la capa de lógica, `GameScene` previamente inyectó su propia referencia en `Board` mediante `Board.Initialize(this)` al momento de su creación, estableciendo un canal bidireccional controlado. La lógica accede a los datos mediante acceso directo a propiedades de `Tile` y `Piece`, o mediante consultas al diccionario estático de `PiecesData` por ejemplo.

6.2. Módulos o componentes del sistema

El sistema se divide en cinco módulos con responsabilidades claramente delimitadas. La Figura 6.5 muestra las dependencias entre ellos. Se representan los componentes principales de cada módulo; los detalles de implementación se describen en el texto que sigue.

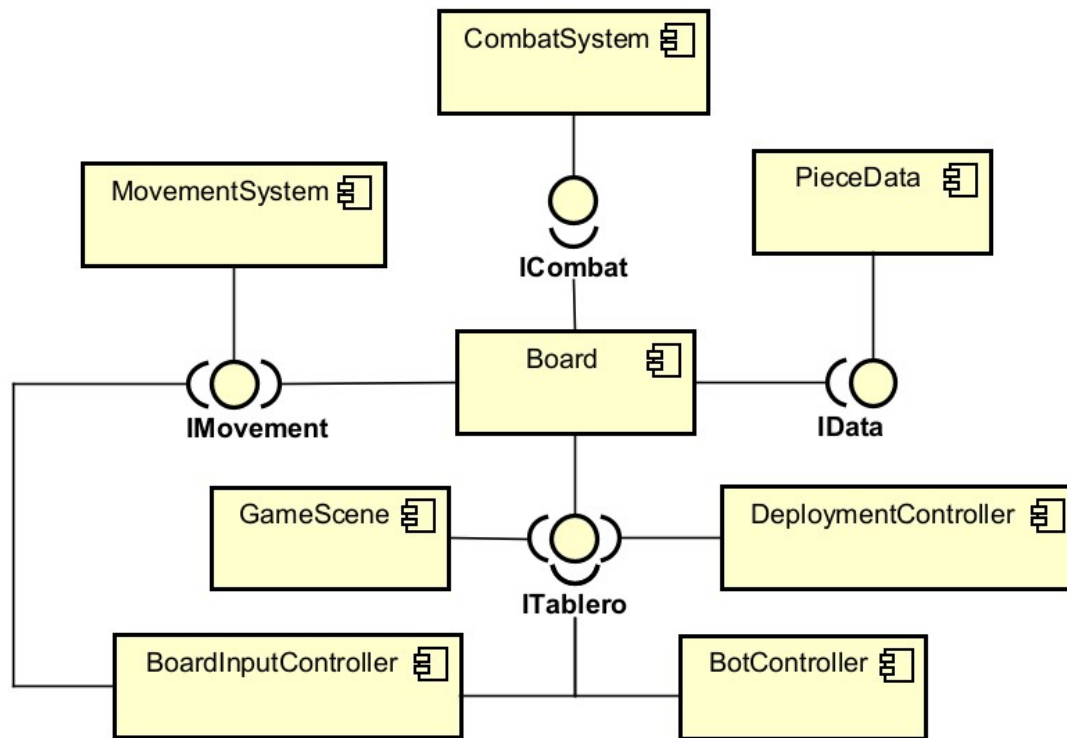


Figura 6.5: Módulos del sistema y sus dependencias principales.

Módulo de orquestación. Formado únicamente por `GameScene`, que actúa como gestor central del estado de la partida. Mantiene el turno activo y el estado mediante el enum `GameState` —con los valores `DEPLOYMENT`, `WAITING_INPUT`, `EXECUTING_ACTION`, `COMBAT` y `GAME_OVER`— y coordina el resto de módulos. En su método `_Ready()` instancia `Board`, `BotController` y `DeploymentController`, inyectando las referencias necesarias entre ellos. También gestiona la pausa mediante `GetTree().Paused`, mecanismo nativo de Godot que detiene el procesamiento de todos los nodos cuyo `ProcessMode` no esté configurado explícitamente como `Always`.

Módulo de tablero. Formado por `Board`, `Tile` y `BoardInputController`. `Board` genera la cuadrícula a partir del `TileMapLayer` en su método `_Ready()`, e instancia `BoardInputController` en `Initialize()`. Es el punto de entrada de todo el input del tablero: cuando el jugador hace clic sobre una casilla, `Tile._InputEvent()` delega en `Board.OnTileClicked()`, que según el estado de la partida redirige la acción bien a `DeploymentController.TryTogglePiece()` durante el despliegue, bien a `BoardInputController.HandleTileClick()` durante la partida. `Board` también expone `MovePiece()` y `ResolveCombat()`, los dos métodos que modifican el estado físico del tablero.

Módulo de piezas. Compuesto por `Piece` y `PiecesData`. `Piece` encapsula tanto los datos de estado (tipo, rango, propietario, visibilidad) como la lógica visual: actualización del sprite mediante regiones del atlas y animaciones de movimiento y revelación. Gestiona además el historial de movimientos necesario para la restricción de oscilación entre 2 casillas. `PiecesData` centraliza todas las definiciones de piezas en un diccionario estático indexado por `PieceType`, evitando que valores de rango, cantidad máxima o columna de atlas queden dispersos por el código.

Módulo de sistemas de juego. Formado por las clases estáticas `MovementSystem` y `CombatSystem`. `MovementSystem` encapsula toda la lógica de validación de movimiento, incluyendo

el movimiento especial del explorador (`IsScoutPathValid()`) y la restricción de oscilación entre dos casillas. `CombatSystem` aplica las reglas de combate en un único método `Resolve()`, con los casos especiales —torreta y phantom— resueltos mediante guardas explícitas antes de la regla general de rango. Al ser clases estáticas sin estado, pueden invocarse desde cualquier parte del código sin efectos secundarios, lo que resulta especialmente valioso de cara a la simulación de partidas en la iteración 4.

Módulo de bot y despliegue. Formado por `BotController` y `DeploymentController`, ambas clases C# puras creadas por `GameScene`. `BotController` recibe una referencia a `Board` en su constructor y, cuando `GameScene` invoca `PlayTurn()`, obtiene la lista de acciones posibles mediante `Board.GetAllPossibleActions()`, selecciona una y la ejecuta mediante `Board.ExecuteBotAction()`. `DeploymentController` se suscribe a los eventos de `DeploymentUI` y gestiona tanto la colocación manual de piezas del jugador (a través de `TryTogglePiece()`, invocado por `Board`) como la generación aleatoria del ejército del bot y el inicio de la partida mediante `StartBattle()`.

6.3. Diseño de clases

6.4. Diseño de interfaz de usuario

6.5. Patrones de diseño aplicados

6.6. Seguridad y rendimiento

Implementación

- 7.1. Organización del código
- 7.2. Licencias requeridas
- 7.3. Batería de pruebas

Capítulo 8

Conclusiones y Líneas futuras

Apéndice A

Manual de instalación

Apéndice B

Manual de usuario

Apéndice C

Apéndice D

Bibliografía

- [1] J. M. Sadurní. El juego real de ur, un entretenimiento en la antigua mesopotamia. https://historia.nationalgeographic.com.es/a/el-juego-real-de-ur-un-entretenimiento-en-la-antigua-mesopotamia_19167.
- [2] Brooklyn Museum. Gaming board inscribed for amenhotep iii with separate sliding drawer. https://web.archive.org/web/20160307212542/https://www.brooklynmuseum.org/opencollection/objects/3536/Gaming_Board_Inscribed_for_Amenhotep_III_with_Separate_Sliding_Drawer.
- [3] Chess Variants. Dou shou qi (animal chess). <https://www.chessvariants.org/other.dir/animal.html>.
- [4] British Antique Dealers' Association. L'attaque board. <https://collections.vam.ac.uk/item/O26365/lattaque-the-famous-game-of-board-game/>.
- [5] Ancient Chess. How to play the chinese land battle game luzhanqi. <https://ancientchess.com/page/play-luzhanqi.htm>.
- [6] Newzoo. Global games market report 2025. https://investgame.net/wp-content/uploads/2025/09/2025_Newzoo_Free_Global_Games_Market_Report.pdf, 2025. Consultado en abril de 2026.
- [7] Marc Palaus, Raquel Viejo-Sobera, Diego Redolar-Ripoll, and Elena M. Marrón. Cognitive enhancement via neuromodulation and video games: Synergistic effects? *Frontiers in Human Neuroscience*, <https://doi.org/10.3389/fnhum.2020.00235>, 2020. Consultado en abril de 2026.
- [8] Alejo García-Naveira, Martín Jiménez Toribio, Borja Teruel Molero, and Alejandro Suárez. Beneficios cognitivos, psicológicos y personales del uso de los videojuegos y esports: una revisión. *Revista de Psicología Aplicada al Deporte y al Ejercicio Físico*, <https://www.revistapsicologiaaplicadadeporteyejercicio.org/art/rpadef2018a15>, 2018. Consultado en abril de 2026.
- [9] Erick Solano-Fernández and Diego Porras-Alfaro. El modelo iterativo e incremental para el desarrollo de la aplicación de realidad aumentada Amón_RA. *Tecnología en Marcha*, vol. 33, n.º 5, https://revistas.tec.ac.cr/index.php/tec_marcha/article/download/5518/5233/15939, 2020. Consultado en abril de 2026.

- [10] British Museum. The royal game of ur. https://www.britishmuseum.org/collection/object/W_1928-1009-378.
- [11] Danielle Zwarg. Senet and twenty squares: Two board games played by ancient egyptians. <https://www.metmuseum.org/es/perspectives/ancient-egypt-board-games>.
- [12] Colin Stapczynski. History of chess | from early stages to magnus. <https://www.chess.com/article/view/history-of-chess>.
- [13] British Antique Dealers' Association. L'attaque game. <https://www.bada.org/object/la-ttaque-game>.
- [14] History of stratego. <https://www.ultraboardgames.com/stratego/history.php>.
- [15] Epic Games. Unreal engine 5 – features. <https://www.unrealengine.com/unreal-engine-5>, 2025. Consultado en abril de 2026.
- [16] Game Developer staff. Unity apologizes to devs, reveals updated Runtime Fee policy. Game Developer, <https://www.gamedeveloper.com/business/unity-apologizes-to-devs-reveals-updated-runtime-fee-policy>, 2023. Consultado en abril de 2026.
- [17] Unity Technologies. A message to our community: Unity is canceling the Runtime Fee. Unity Discussions, <https://discussions.unity.com/t/a-message-to-our-community-unity-is-canceling-the-runtime-fee/1517714>, 2024. Consultado en abril de 2026.
- [18] Unity Technologies. Unity – real-time development platform. <https://unity.com/solutions/game>, 2025. Consultado en abril de 2026.
- [19] Godot Engine. Godot engine – features. <https://godotengine.org/features/>, 2025. Consultado en abril de 2026.
- [20] Tyler Wilde. Slay the spire 2 ditched Unity for open-source engine Godot after over 2 years of development. PC Gamer, <https://www.pcgamer.com/games/card-games/slay-the-spire-2-ditched-unity-for-open-source-engine-godot-after-2-years-of-development/>, 2024. Consultado en abril de 2026.
- [21] David Cailleteau. Slay the spire 2 had a massive launch with an estimated 4.6M copies sold, over \$92M in revenue so far. Wccftech, <https://wccftech.com/slay-the-spire-2-estimated-4-6-million-copies-sold-92-million-revenue-generated/>, 2026. Consultado en abril de 2026.
- [22] Glassdoor. Sueldo: Junior game developer en españa. https://www.glassdoor.es/Sueldos/junior-game-developer-sueldo-SRCH_K00,21.htm, 2025. Consultado en abril de 2026.